

Quantized Transformer-Based Dependency-Aware Computation Offloading for UAV-Assisted IoT Edge-Cloud Computing

Parth Srivastava, Sateesh Kumar Peddoju, Ananya Kapoor, Aashu Raj, and Subal Aggarwal

Abstract—Unmanned Aerial Vehicle (UAV)-assisted Mobile Edge Computing (MEC) has emerged as an effective solution for supporting latency-sensitive Internet of Things (IoT) applications in infrastructure-limited environments. By extending Edge computing capabilities to aerial platforms, UAV-enabled systems provide flexible and on-demand computational support for distributed IoT devices. However, practical deployment of intelligent computation offloading strategies remains challenging due to limited onboard computational resources, strict energy constraints, and complex task dependencies commonly observed in real-world IoT workloads. In many applications, computational tasks exhibit directed acyclic graph (DAG) structures, requiring dependency-aware scheduling to ensure correct execution order and efficient resource utilization. This work proposes a quantized Transformer-based dependency-aware computation offloading framework for UAV-assisted IoT Edge-Cloud computing systems. The proposed approach models IoT workloads as DAGs and employs a lightweight Transformer encoder within a reinforcement learning actor-critic architecture to capture contextual relationships among task dependencies, wireless conditions, and computing resources. To enable practical deployment on resource-constrained UAV platforms, the trained Transformer policy is further compressed using INT8 post-training quantization, significantly reducing model storage requirements and runtime memory usage while maintaining decision quality. Extensive simulations demonstrate that, compared with the UDCO baseline framework, the proposed framework reduces overall system energy consumption by 23.63% and UAV energy consumption by 23.62%, while achieving a 1.04% reduction in average task latency. Furthermore, model compression reduces storage requirements by 59.8% and runtime memory usage by 18%, enabling efficient deployment on hardware-constrained UAV platforms.

Index Terms—Mobile Edge Computing (MEC), UAV, computation offloading, Transformers, reinforcement learning, quantization, INT8.

I. INTRODUCTION

The rapid growth of the Internet of Things (IoT) is increasing the demand for low-latency computing infrastructures to support applications such as industrial monitoring, intelligent transportation systems, environmental sensing, and disaster management [1]. Although cloud computing platforms provide abundant computational capacity, their centralized architecture

can incur significant communication delay when data must be transmitted from end devices to remote data centres, which may limit the responsiveness of latency-sensitive IoT services [2]. Mobile Edge Computing (MEC) mitigates this limitation by moving computation closer to end devices, thereby reducing transmission latency and alleviating backbone network congestion [2]. However, most MEC deployments rely on fixed infrastructure (e.g., base stations or dedicated Edge servers), which may be unavailable or disrupted in disaster-affected regions, remote facilities, or large outdoor environments [3]. Unmanned Aerial Vehicles (UAVs) can extend MEC to infrastructure-limited areas by acting as aerial Edge servers that provide flexible, on-demand computation with rapid deployment and favourable line-of-sight links [3], [4]. UAV-assisted MEC can therefore support emergency response, environmental monitoring, precision agriculture, and temporary communication networks [3].

Despite these benefits, UAV platforms remain constrained by battery capacity, payload, and onboard processing capability, making energy-aware resource management essential [3], [5]. Moreover, many IoT applications comprise multiple interdependent subtasks that can be represented as directed acyclic graphs (DAGs); these dependencies require scheduling and offloading policies that respect execution order while adapting to available resources [6], [7]. In UAV-assisted MEC, computation offloading becomes a sequential decision problem that depends on time-varying wireless conditions, task dependencies, and heterogeneous resource availability [5], [8]. Deep reinforcement learning (DRL) has therefore been explored to learn adaptive offloading and trajectory-control strategies in dynamic environments [4], [9], [10]. However, conventional DRL policies that encode system states as fixed-dimensional representations may struggle to capture the contextual relations induced by DAG-structured workloads and coupled constraints, motivating richer sequence models for policy learning [11], [12]. Transformer architectures provide a promising alternative by using self-attention to model long-range interactions and structured dependencies [13]. Unlike conventional multilayer perceptron policies that process state variables largely independently, the self-attention mechanism enables the learning model to capture cross-feature relationships among task dependencies, wireless conditions, UAV mobility, and resource availability, thereby supporting more context-aware offloading decisions. Recent studies have begun to apply Transformer-enhanced reinforcement learning to scheduling and resource-management tasks in Edge-Cloud

Parth Srivastava, Sateesh Kumar Peddoju, and Aashu Raj are with the Indian Institute of Technology Roorkee, Roorkee, India (e-mail: pr_srivastav@cs.iitr.ac.in; sateesh@cs.iitr.ac.in; aashuraj9097@gmail.com).

Ananya Kapoor is with the College of Engineering Roorkee, Roorkee, India (e-mail: kapoorananya2003@gmail.com).

Subal Aggarwal is with Doon University, Dehradun, India (e-mail: subal-aggarwal525@gmail.com).

settings, indicating potential benefits for dependency-aware decision making [11], [12]. Nevertheless, the computational and memory overhead of Transformers can hinder direct deployment on UAV platforms with limited hardware resources [3].

Motivation: In practical UAV-assisted MEC deployments, IoT applications often follow dependency-aware task structures such as multi-stage video analytics, aerial surveillance pipelines, and disaster monitoring workflows, where intermediate outputs must be processed sequentially across multiple computation stages. Consequently, the offloading policy must simultaneously account for DAG task dependencies together with strict energy and computation budgets on UAV-mounted Edge platforms [3], [5], [6]. While Transformer architectures can effectively encode such contextual relationships, their computational and memory footprint may exceed the capabilities of UAV Edge hardware [13]. Therefore, lightweight compression techniques, such as INT8 post-training quantization, are required to enable efficient real-time inference without significantly degrading decision quality [14]–[16].

To address these challenges, we propose a lightweight Transformer-based reinforcement learning framework for dependency-aware task offloading in UAV-assisted Edge–Cloud systems. The framework models IoT applications as DAG workloads and employs a lightweight Transformer encoder within an actor–critic reinforcement learning architecture to learn context-aware offloading decisions, while INT8 post-training quantization reduces model size and runtime memory requirements for deployment on UAV platforms [14], [15]. The main contributions of this work are summarized as follows:

- 1) *UAV-Assisted Device–Edge–Cloud Collaborative Computing Framework:* We develop a UAV-assisted device–Edge–Cloud (D–E–C) collaborative computing architecture in which IoT devices execute DAG-based applications while tasks are dynamically distributed across local devices, UAV-mounted Edge servers, and a remote Cloud. The framework incorporates UAV trajectory dynamics, wireless communication constraints, and models UAV flight and hover energy consumption to enable realistic system operation.
- 2) *Dependency-aware Computation Offloading Model:* We formulate the computation offloading problem for DAG-structured IoT workloads by modelling task dependencies, communication constraints, and heterogeneous computing resources. The formulation jointly considers task completion latency, system energy consumption, and UAV operational constraints.
- 3) *Dependency-aware Transformer Reinforcement Learning Policy:* We develop a Transformer-enhanced actor–critic learning architecture that encodes task dependencies, communication conditions, and system resource states using multi-head self-attention. This design enables the learning policy to capture long-range interactions among tasks within DAG workflows while adapting offloading decisions to dynamic UAV mobility and wireless conditions.
- 4) *Lightweight Deployment via INT8 Quantized Transform-*

TABLE I: Comparison of UAV-assisted MEC task offloading approaches

Authors	Method	Strength	Limitation
Wang et al. [4]	SAC-TMA (DRL)	Joint trajectory, power and association optimization	No temporal modelling
Yang et al. [7]	DAGOA + SAC	Hierarchical trajectory and offloading control	Complex training
Zakaryia et al. [9]	MADDPG + DTLCM	Multi-UAV cooperative optimization	Scalability challenges
Li et al. [10]	MSE-PPO DRL	Energy–latency fairness optimization	High-dimensional action space
Peng et al. [17]	EDRA optimization	Energy-efficient resource allocation	Limited task modelling
Xu et al. [18]	MATD3-TORA DRL	Joint trajectory and resource optimization	No DAG task modelling

ers: To enable deployment on resource-constrained UAV hardware, we compress the trained Transformer policy using INT8 post-training quantization, reducing model storage requirements and runtime memory usage while preserving decision accuracy.

- 5) *Comprehensive Performance Evaluation:* We conduct extensive simulations to evaluate the proposed framework in dynamic UAV-assisted MEC environments. The results demonstrate improved system energy efficiency and reduced UAV energy consumption while maintaining low task completion latency compared with existing learning-based offloading approaches.

Such a framework is valuable for infrastructure-limited scenarios requiring reliable computation support, including disaster response operations, remote industrial monitoring, large-scale environmental sensing, and precision agriculture systems. The remainder of this paper is organized as follows. Section II reviews related work; Section III presents the system model and problem formulation; Section IV describes the proposed Transformer-based offloading framework and INT8 quantization; Section V evaluates the performance through simulations; and Section VI concludes the paper.

II. RELATED WORKS

Computation offloading and resource management in UAV-assisted mobile Edge computing have attracted significant research interest in recent years. Existing studies can generally be grouped into three categories: learning-based task offloading in UAV-assisted MEC systems, Transformer-based approaches for scheduling and resource allocation in Edge or Cloud environments, and quantization techniques that enable efficient deployment of Transformer models on resource-constrained platforms. This section reviews representative studies in these directions and highlights their limitations.

A. Learning-Based Task Offloading in UAV-Assisted MEC Systems

Wang et al. [4] proposed a deep reinforcement learning-based SAC-TMA framework for joint optimisation of UAV trajectory, user association, and transmit power in UAV-assisted MEC systems. The approach improves coordinated resource management but does not explicitly capture temporal system

TABLE II: Comparison of Transformer-based approaches in MEC and Edge computing

Authors	Method	Strength	Limitation
Wang et al. [11]	TF-DDRL (Transformer + DRL)	Efficient DAG scheduling	Not UAV-MEC specific
Pourakbar et al. [19]	Transformer-based AIaaS offloading	Energy-efficient Edge inference	Limited scalability, no UAV support
Ndiaye et al. [20]	DTQN-based RL for VNF/SFC placement	QoS-aware service placement	Static MEC nodes
Nandagopal et al. [21]	HCS-Transformer task scheduling	Energy-efficient scheduling	Focused on centralized Cloud
Li et al. [12]	GTrXL-PR (Transformer + Meta-RL)	Captures long-term dependencies	High training complexity

dynamics. Yang et al. [7] introduced a hybrid DAGOA–SAC framework that combines genetic optimization for task offloading with soft actor critic learning for UAV trajectory control. The hierarchical structure improves task completion efficiency but increases training complexity. Zakaryia et al. [9] proposed a multi-UAV MEC framework integrating MADDPG with a task location matching mechanism to jointly optimize trajectory, task allocation, and resource management. Although the approach improves system efficiency, scalability remains challenging in large UAV networks. Li et al. [10] developed an actor–critic based MSE-PPO framework that jointly optimizes UAV trajectory and task offloading while ensuring service fairness among users. The method improves energy and latency performance but introduces training complexity due to the high-dimensional action space. Peng et al. [17] proposed an Efficient Dynamic Resource Allocation algorithm using stochastic optimization for UAV-assisted MEC systems. The approach reduces system energy consumption but assumes simplified task structures. Xu et al. [18] introduced a multi-agent DRL framework (MATD3-TORA) for joint optimization of UAV trajectory, CPU allocation, and task scheduling. Although the method reduces latency and energy consumption, it does not consider DAG-based task dependencies.

B. Transformer-Based Scheduling in Edge and Cloud Computing

Wang et al. [11] proposed TF-DDRL, a Transformer enhanced distributed deep reinforcement learning framework for scheduling DAG-based IoT applications in Edge Cloud environments. By integrating Transformer representations with distributed actor critic learning, the method improves scheduling efficiency and convergence speed. However, it does not consider UAV-assisted MEC systems. Pourakbar et al. [19] introduced a Transformer-based AI as a Service offloading framework for Edge inference tasks. The approach models inference workloads as DAG workflows and reduces mobile device energy consumption while meeting latency constraints. However, it focuses on Edge AI inference and does not address UAV-assisted MEC environments. Ndiaye et al. [20] proposed a Transformer-based reinforcement learning framework for virtual network function placement in MEC networks. The

TABLE III: Transformer quantization techniques relevant for Edge inference

Authors	Method	Strength	Limitation
Kim et al. (2021) [16]	I-BERT	Fully INT8 inference without floating point	Requires specialized integer kernels
Lin et al. (2021) [22]	FQ-ViT	Fully quantized vision transformer	Limited to vision models
Xiao et al. (2023) [15]	SmoothQuant	Accurate INT8 quantization for LLMs	Designed mainly for large language models
Lin et al. (2024) [23]	AWQ	Hardware-friendly low-bit weight quantization	Requires activation statistics for calibration
Yao et al. (2022) [24]	ZeroQuant	Scalable PTQ with minimal accuracy loss	Requires optimized inference backend
Dettmers et al. (2022) [14]	LLM.int8()	Efficient mixed-precision quantization	Requires handling activation outliers

approach improves QoS-aware service placement and resource utilization but assumes static MEC nodes. Nandagopal et al. [21] developed a Hybrid Cuckoo Search Transformer framework for energy-aware task scheduling in Cloud computing environments. Although the method improves energy efficiency and resource utilization, it primarily targets centralized Cloud systems rather than distributed MEC environments. Li et al. [12] introduced GTrXL-PR, which combines Gated Transformer XL with meta reinforcement learning for task offloading in MEC systems. The model captures long-term dependencies and improves adaptability in dynamic environments, but increases training complexity.

C. Transformer Quantization for Efficient Edge Deployment

Kim et al. [16] proposed I-BERT, an integer-only quantization framework for Transformer models that enables fully INT8 inference without floating-point operations. The method preserves model accuracy while improving inference efficiency, although it requires specialized integer kernels. Lin et al. [22] introduced FQ-ViT, a post-training quantization technique for vision transformers using integer-only operations. The method achieves near-lossless accuracy but is designed primarily for vision-based transformer architectures. Xiao et al. [15] proposed SmoothQuant, which enables efficient INT8 quantization for large language models by shifting quantization difficulty from activations to weights through channel-wise scaling. Although highly effective for large models, it mainly targets LLM architectures. Lin et al. [23] developed Activation-aware Weight Quantization (AWQ), a hardware-friendly method that enables low-bit weight quantization while preserving model generalization. However, the method requires offline activation statistics for calibration. Yao et al. [24] proposed ZeroQuant, a scalable post-training quantization framework that combines hardware-friendly quantization with layer-wise knowledge distillation to maintain model accuracy. The approach requires customized inference backends for optimal performance. Dettmers et al. [14] introduced LLM.int8(), a mixed-precision quantization approach that enables accurate INT8 inference for very large transformer models while significantly reducing memory usage. The method requires careful handling of activation outliers during quantization.

Research Gap: Despite significant progress in UAV-assisted MEC and learning-based task offloading, several important

challenges remain. Existing DRL-based offloading frameworks typically employ conventional neural network architectures such as multilayer perceptrons that process input features largely independently and therefore struggle to capture complex contextual relationships among task dependencies, wireless conditions, and resource constraints in DAG-based workloads. Although recent studies have explored Transformer-based models for scheduling and resource management in edge or cloud computing environments, these approaches generally assume static infrastructure and do not consider UAV-assisted MEC systems characterized by mobility, energy limitations, and dynamic wireless links. Moreover, the high computational and memory overhead of Transformer architectures makes their direct deployment on resource-constrained UAV platforms difficult without additional model optimization. In addition, most UAV-assisted MEC studies assume independent tasks or simple queue-based workloads, whereas practical IoT applications frequently consist of DAG-structured tasks with strict precedence constraints. Efficiently modelling these dependency relationships while simultaneously handling dynamic wireless conditions and UAV mobility therefore remains an open challenge. Consequently, existing solutions cannot simultaneously achieve dependency-aware scheduling, adaptive learning under mobility, and deployment-efficient model design suitable for UAV hardware. These limitations motivate the development of a lightweight Transformer-based offloading framework that can capture dependency-aware task relationships while remaining computationally feasible for deployment on UAV-mounted edge platforms.

III. SYSTEM ARCHITECTURE AND OPTIMIZATION FRAMEWORK

Fig. 1 illustrates the UAV-assisted Edge-Cloud computation offloading architecture considered in this work. The system consists of three computing layers: local IoT devices ($e = 0$), a UAV-mounted Edge server ($e = 1$), and a remote Cloud server ($e = 2$). The UAV operates within a predefined service region and provides Edge computing support to IoT devices generating independent as well as dependency-structured tasks. The trained Transformer-based actor-critic model is deployed onboard the UAV for real-time offloading decision inference. The total simulation time t_{sim} is divided into T frames, indexed as $\tau \in \{1, 2, \dots, T\}$. After each frame, IoT devices randomly change position and check for Line-of-Sight (LoS) or Non-Line-of-Sight (NLoS) connectivity with the UAV. Based on this, the UAV applies the computation offloading (CO) policy: tasks are either processed locally, at the UAV, or forwarded to the Cloud if neither can execute them efficiently. The system formulation follows the general UAV-assisted device-Edge-Cloud collaborative computing paradigm widely adopted in prior dependency-aware offloading studies, while the present work extends this foundation through Transformer-based policy learning and lightweight quantized deployment [6].

A. System Assumptions and Operating Conditions

The proposed framework considers a UAV-assisted Edge-Cloud computing system as illustrated in Fig. 1. IoT

TABLE IV: Table of Notations

Symbol	Description
$DAG(k)$	DAG structure of the k th application
UE_u	u^{th} IoT device
u	user index
$P_u(\tau)$	Position of user at time frame τ
$P_a(\tau)$	Position of UAV-MEC at time frame τ
$g_{u,a}(\tau)$	Channel gain value b/w UAV-MEC & IoT user at time frame τ
$g_{a,s}(\tau)$	Channel gain value b/w UAV-MEC & Cloud server at time frame τ
$l_{k,m}$	Calculation data of $task_k^m$
$T_{u,a}^{tx}(k, m)$	Transmission latency b/w UE_u & UAV-MEC
$T_a^{comp}(k, m)$	Calculation of time delay on UAV-MEC
$r_{u,a}(\tau), r_{a,s}(\tau)$	transmission rate from UE_u to UAV and from UAV to Cloud
$T_s^{comp}(k, m)$	Calculation of time delay on Cloud server
$T_{a,s}^{tx}(k, m)$	Transmission latency b/w UAV-MEC & Cloud server
$\tau_{k,m}^s$	Start time of $task_k^m$
$\tau_{k,m}^e$	End time of $task_k^m$

devices are distributed within a predefined service region and communicate with the UAV through wireless links when they lie within its coverage area. The UAV acts as a mobile Edge server capable of processing tasks locally or forwarding them to a remote Cloud server through a relatively stable backhaul link. Computation tasks generated by IoT devices may therefore be executed locally at the device, processed at the UAV-mounted Edge server, or relayed to the Cloud depending on the offloading decision.

The UAV moves within the service region across discrete time frames and consumes energy for both flight and hovering operations. Wireless channels between IoT devices and the UAV vary over time due to user mobility and environmental conditions, whereas the UAV-Cloud backhaul link is assumed to remain stable. To ensure practical deployment on resource-constrained platforms, the reinforcement learning policy used for computation offloading is trained offline using high-performance computing infrastructure, while only the inference stage of the trained model is executed onboard the UAV during operation. The CO process, therefore, consists of four stages: (i) task transmission from IoT device to UAV, (ii) execution at the UAV Edge server, (iii) optional forwarding from UAV to Cloud, and (iv) Cloud-side computation. Because the computation results are typically small and the UAV-Cloud downlink is reliable, the return transmission cost is assumed negligible.

B. Task Model and Dependency Structure

IoT application workloads are modelled using Directed Acyclic Graphs (DAGs) [5], where nodes represent subtasks and Edges define precedence constraints among them. For the k -th application,

$$DAG(k) = \{\text{Task}(k), \text{Con}(k)\}, \quad (1)$$

where $\text{Task}(k) = \{\text{task}_k^1, \dots, \text{task}_k^m\}$ denotes the set of subtasks and $\text{Con}(k)$ represents the dependency relations

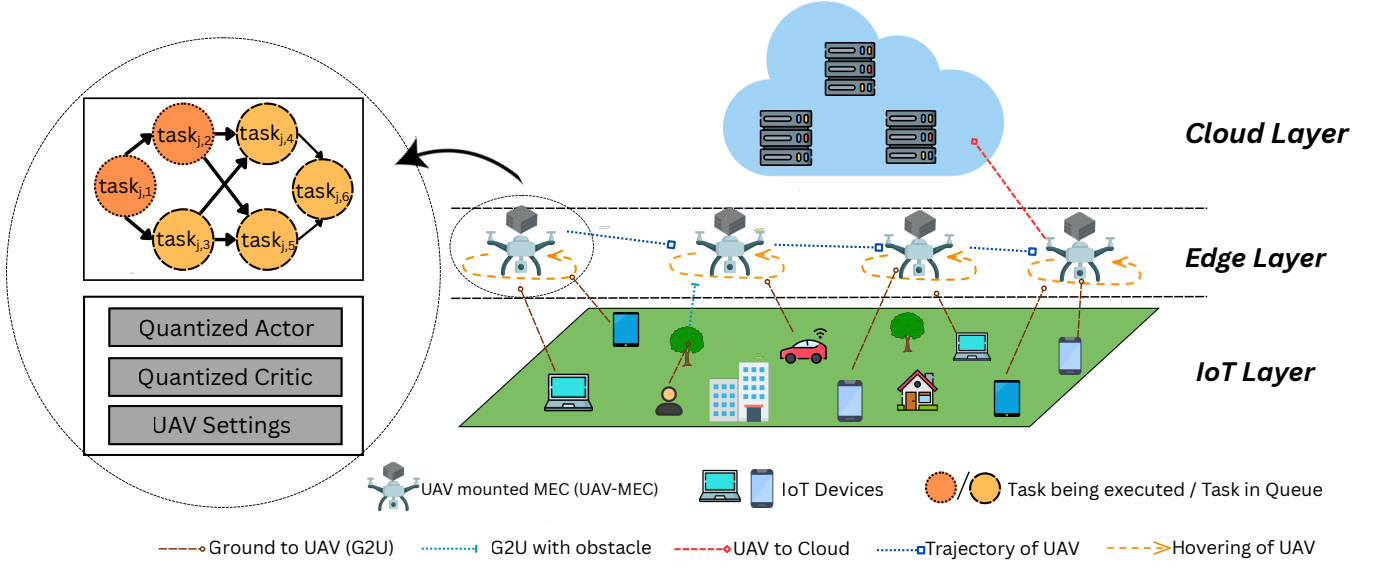


Fig. 1: Architecture of the proposed UAV-assisted Edge-Cloud computing framework.

determining their execution order. A node without incoming edges corresponds to a start task, while nodes without outgoing edges correspond to end tasks.

Each task is represented as

$$\text{task}_k^m = \{\text{UE}_u, b_{k,m}, l_{k,m}, \tau_{k,m}^s, \tau_{k,m}^e\},$$

where $b_{k,m}$ denotes the input data size, $l_{k,m}$ represents the required computation workload, and $\tau_{k,m}^s$ and $\tau_{k,m}^e$ indicate the start and completion times of the task.

C. Task Arrival and Dependency Behaviour

IoT devices generate computation tasks at the beginning of each frame following a stochastic arrival process that reflects the bursty and time-varying behaviour of practical IoT workloads. The number and type of arriving tasks may differ across users, producing dynamic computational demand over time. Each application follows the DAG structure defined above, meaning that a subtask can begin execution only after all its predecessor subtasks have completed. If any parent task remains unfinished, the corresponding child task becomes temporarily blocked and cannot be scheduled for local processing or offloading during that frame. This dependency-aware blocking mechanism preserves the correct execution order and prevents incomplete or invalid computations. Consequently, system delay, energy consumption, and offloading decisions depend not only on wireless conditions and available computing resources but also on the readiness of subtasks within the DAG. Incorporating both dynamic task arrivals and dependency-aware scheduling enables the model

to capture realistic IoT application behaviour while remaining consistent with the modelling principles adopted in the base framework [6].

D. UAV and User Mobility

We consider a UAV-MEC flying at a fixed altitude Z with coordinates $P_a(\tau) = [x_a(\tau), y_a(\tau), Z]$ in a 3D Cartesian system. The u -th IoT device (UE_u) is located at $P_u(\tau) = [x_u(\tau), y_u(\tau), 0]$. Each time frame τ has duration $\Delta\tau$. At time τ , the UAV receives a flight command specifying heading $\theta_a(\tau)$ and speed $\omega_a(\tau)$, as shown in Fig. 2. The UAV position updates as:

$$x_a(\tau + 1) = x_a(\tau) + d_a(\tau) \cos(\theta_a(\tau)), \quad (2)$$

$$y_a(\tau + 1) = y_a(\tau) + d_a(\tau) \sin(\theta_a(\tau)), \quad (3)$$

where $d_a(\tau) = \omega_a(\tau)(\Delta\tau)$. If UE_u moves randomly by $d_u(\tau)$, its position becomes:

$$P_u(\tau+1) = \begin{bmatrix} x_u(\tau) + d_u(\tau) \cos(\theta_u(\tau)) \\ y_u(\tau) + d_u(\tau) \sin(\theta_u(\tau)) \\ 0 \end{bmatrix}. \quad (4)$$

The UAV and UE must remain within the service area $[0, L_s] \times [0, W_s]$:

$$0 \leq x_a(\tau) \leq L_s, \quad 0 \leq y_a(\tau) \leq W_s, \quad (5)$$

$$0 \leq x_u(\tau) \leq L_s, \quad 0 \leq y_u(\tau) \leq W_s. \quad (6)$$

Angles and speeds satisfy:

$$0 \leq \theta_a(\tau), \theta_u(\tau) \leq 2\pi, \quad 0 \leq \omega_a(\tau) \leq \omega_{\max}. \quad (7)$$

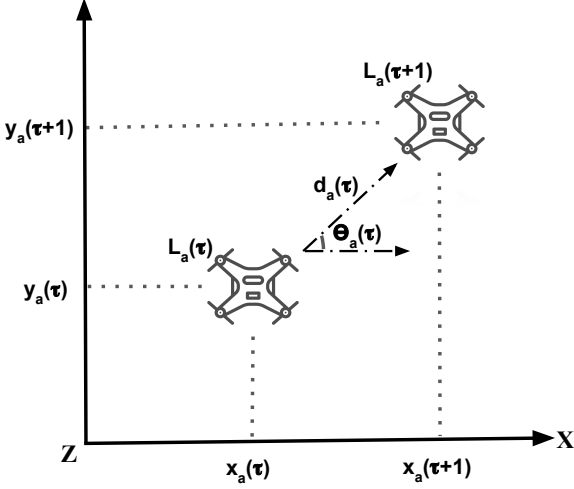


Fig. 2: UAV and IoT device mobility model.

The UAV flight energy consumption is:

$$E_f(\tau) = 0.5 \omega_a^2(\tau) m_a d_a(\tau), \quad (8)$$

where m_a is UAV mass. Hovering energy [25] is:

$$E_h(\tau) = n_w \sqrt{\frac{(m_a g)^3}{2 \rho \pi R_w^2}}, \quad (9)$$

with n_w the number of wings, R_w their radius, ρ air density, and g the gravitational constant. Given the UAV/UE trajectories above, the next step is to derive a rate model suitable for offloading analysis between a UE and the UAV.

UAV Trajectory Bounds: The UAV operates within a fixed rectangular service region and updates its heading and velocity at each frame, subject to maximum speed and manoeuvrability constraints. These limits prevent unrealistic behaviour such as abrupt turns, instantaneous acceleration, or position jumps, ensuring physically feasible flight dynamics. When approaching the boundary of the service region, the UAV is redirected to remain within the operational area and maintain coverage of IoT devices. These trajectory constraints influence communication distance, LoS probability, energy consumption, and available offloading opportunities, thereby reflecting practical UAV deployment conditions consistent with the base framework.

UAV Battery and Energy Behavior: The UAV operates with a limited onboard battery whose energy is consumed through propulsion during movement and hovering while stationary. Propulsion cost depends on heading and velocity commands, whereas hovering represents the baseline power required to remain airborne. Energy consumption accumulates across frames, determining the duration for which the UAV can provide MEC services. Each episode therefore begins with a fixed energy budget, and the reinforcement learning policy penalizes low battery levels or inefficient movement. This encourages energy-aware trajectory planning and efficient task offloading, consistent with the modelling assumptions of the base framework [6].

E. Wireless Communication and Connectivity

UE_u communicates with the UAV-MEC via a wireless uplink, typically under Line-of-Sight (LoS) conditions. From the mobility model, their 3D Euclidean distance is:

$$d_{u,a}(\tau) = \sqrt{\|P_a(\tau) - P_u(\tau)\|^2 + Z^2}. \quad (10)$$

To ensure connectivity, the link must satisfy:

$$d_{u,a}(\tau) \leq d_{\max}, \quad \forall u \in \mathcal{U}_\tau. \quad (11)$$

User Mobility Behaviour: IoT users are mobile, and their movement directly affects link quality and offloading performance. Each user moves within the service region following a moderate random-walk pattern with periodically updated speed and direction, representing pedestrians or slow-moving devices. Movement toward the UAV strengthens the link, while movement away or toward the boundary reduces link quality. When a user reaches the service boundary, it is redirected to remain within the UAV's coverage area. These mobility dynamics cause fluctuations in distance, LoS/NLoS probability, and pathloss, which influence transmission rates, delay, and energy consumption. Incorporating user mobility ensures realistic communication dynamics consistent with the modelling approach of the base framework.

Connectivity Constraints: A UE can offload tasks only when it lies within the UAV's communication range. If the UE-UAV distance exceeds the maximum service radius, the link becomes invalid, and transmission is suspended until the device re-enters the coverage region. This constraint prevents unrealistic long-distance communication and introduces natural delays consistent with UAV-ground systems. Connectivity variations also influence task prioritization, since users closer to the UAV experience stronger links, while those near the boundary face weaker rates or temporary outages. These variations affect offloading decisions, energy usage, and LoS/NLoS behaviour, ensuring realistic system operation consistent with the base framework.

LoS/NLoS Propagation Behaviour: UAV-ground communication alternates between Line-of-Sight (LoS) and Non-Line-of-Sight (NLoS) depending on elevation angle, user position, and environmental obstacles. Higher elevation angles and shorter distances increase the likelihood of LoS, while obstacles and low viewing angles lead to NLoS conditions. The system therefore adopts an environment-dependent probabilistic model (e.g., suburban, urban, dense urban) to determine the link condition at each frame. LoS provides lower attenuation and stronger signal quality, whereas NLoS introduces additional loss and noise, reducing transmission rates and increasing delay and energy consumption. This probabilistic propagation model captures realistic wireless variability and follows the base framework [6].

Pathloss Characteristics in UAV-Device Communication: The wireless channel between the UAV and IoT devices experiences distance-dependent attenuation determined by the current LoS or NLoS condition. As the UAV moves, pathloss varies with altitude, horizontal distance, and environmental obstacles. LoS links provide clearer propagation with lower attenuation, whereas NLoS links introduce shadowing and

obstruction losses. The model, therefore, applies separate attenuation behaviours for LoS and NLoS and updates the effective pathloss at each frame. These variations directly affect achievable data rates, communication latency, and transmission energy consumption. Incorporating distinct LoS/NLoS pathloss behaviour ensures realistic modelling of UAV-assisted MEC communication environments [6]. The LoS channel gain [26] is:

$$g_{u,a}(\tau) = \frac{\beta_0}{\|P_a(\tau) - P_u(\tau)\|^2 + Z^2}, \quad (12)$$

where β_0 is the gain per unit distance. In realistic environments, obstacles may cause Non-LoS (NLoS) [8], leading to a transmission rate:

$$r_{u,a}(\tau) = B \log_2 \left(1 + \frac{\mathcal{P}_u^{tx} g_{u,a}(\tau)}{\sigma_{\text{NLoS}}^2 + p_{u,a}(\tau) L_{\text{NLoS}}} \right), \quad (13)$$

where B is bandwidth, $\mathcal{P}_u^{tx}$ is UE transmit power, σ_{NLoS}^2 is noise, L_{NLoS} is NLoS loss, and $p_{u,a}(\tau)$ indicates NLoS presence. For the UAV–Cloud link, the channel gain is:

$$g_{a,s}(\tau) = \frac{\beta_0}{\|P_s(\tau) - P_a(\tau)\|^2 + Z^2}, \quad (14)$$

and with superior LoS, the transmission rate is:

$$r_{a,s}(\tau) = B \log_2 \left(1 + \frac{\mathcal{P}_a^{tx} g_{a,s}(\tau)}{\sigma_{\text{VLoS}}^2} \right), \quad (15)$$

where $\mathcal{P}_a^{tx}$ is UAV transmit power and σ_{VLoS}^2 is LoS noise power.

F. Multi-Tier Computation Offloading

The CO performance is evaluated in terms of delay and energy consumption for local, Edge, and Cloud execution. *CPU Frequency Constraints:* All computing nodes i.e. IoT devices, the UAV, and the Cloud server, operate with fixed upper limits on their CPU cycle frequencies. Each device performs computation within its maximum frequency budget, ensuring that processing delay and energy consumption remain within its hardware capabilities. Since the dynamic power of a processor increases rapidly with higher frequencies, these limits naturally restrict how fast a task can be completed locally, at the UAV, or at the Cloud. The UAV and IoT devices cannot scale their CPU frequency beyond their preset capacities, making computation speed directly tied to these bounds. These constraints also influence offloading decisions: tasks with large workloads may be better executed at the UAV or Cloud when local CPU capacity is insufficient. Incorporating these CPU limits ensures realistic processing behaviour and aligns with the modelling assumptions used in the base framework [6].

1) *Local Computing:* For UE_u, the delay to execute task_k^m locally [27] is:

$$T_u(k, m) = \frac{\phi l_{k,m}}{f_u}, \quad (16)$$

where f_u is CPU frequency and ϕ is cycles/bit. The corresponding energy [28], [29] is:

$$E_u(k, m) = \zeta f_u^2 \phi l_{k,m}, \quad (17)$$

with ζ a chip-dependent constant.

2) *UAV Edge Computing:* If task_k^m is offloaded to the UAV-MEC:

$$T_{u,a}(k, m) = \frac{b_{k,m}}{r_{u,a}(\tau)} + \frac{\phi l_{k,m}}{f_a}, \quad (18)$$

$$E_{u,a}(k, m) = \mathcal{P}_u^{tx} \frac{b_{k,m}}{r_{u,a}(\tau)} + \zeta f_a^2 \phi l_{k,m}, \quad (19)$$

where $b_{k,m}$ is task size, $r_{u,a}(\tau)$ is uplink rate, f_a is UAV CPU frequency, and $\mathcal{P}_u^{tx}$ is UE transmit power.

3) *Cloud Computing:* If further offloaded to the Cloud:

$$T_{u,s}(k, m) = \frac{b_{k,m}}{r_{u,a}(\tau)} + \frac{b_{k,m}}{r_{a,s}(\tau)} + \frac{\phi l_{k,m}}{f_s}, \quad (20)$$

$$E_{u,s}(k, m) = \mathcal{P}_u^{tx} \frac{b_{k,m}}{r_{u,a}(\tau)} + \mathcal{P}_a^{tx} \frac{b_{k,m}}{r_{a,s}(\tau)} + \zeta f_s^2 \phi l_{k,m}, \quad (21)$$

where $r_{a,s}(\tau)$ is UAV–Cloud rate, f_s is Cloud CPU frequency, and $\mathcal{P}_a^{tx}$ is UAV transmit power.

System Cost Definition: The overall system cost captures the combined impact of execution delay and energy consumption across local, UAV, and Cloud processing. Each offloading decision influences both the task completion time and the transmission or computation energy required. Tasks with large workloads or weak wireless links incur higher cost, while favourable connectivity and stronger computing resources reduce it. The cost accumulates across all subtasks within an episode, allowing the reinforcement learning agent to learn policies that minimize delay while conserving UAV battery resources. This unified delay–energy metric balances efficiency and endurance and follows the modelling strategy adopted in the base framework [6].

G. Markov Decision Process Formulation

To jointly optimize computation offloading (CO) and UAV endurance, the problem is formulated as a Markov Decision Process (MDP) and solved using DRL. The MDP consists of the following components:

- *Observation Space:* Includes UAV states (position and remaining energy), IoT device states (locations and task queues), and environmental parameters such as link quality and task dependencies. All observations are normalized before being processed by the neural network.
- *Action Space:* A hybrid space combining discrete decisions (offloading destinations and target devices) and continuous controls (UAV heading and speed). Algorithms such as Soft Actor–Critic (SAC) are suitable for handling this mixed action structure.
- *Reward Function:* Defined as the negative weighted sum of task delay and UAV energy consumption, encouraging policies that reduce both metrics.

System states evolve dynamically as UAV movement, user mobility, task arrivals, and dependency updates modify communication conditions and available computing resources. The learning agent observes these changes and learns an offloading strategy that minimizes the cumulative delay–energy cost.

Episode Definition and Reset Conditions: Each training episode represents one operational cycle of the UAV-assisted MEC system. An episode starts with the UAV at its initial

position, IoT devices randomly distributed within the service region, and all task queues initialized. DAG structures for each user are created with all subtasks initially pending. The environment evolves frame by frame as users move, tasks arrive, and offloading actions are executed. The episode terminates when the UAV battery becomes insufficient, the maximum number of frames is reached, or all tasks are completed, after which the environment resets for the next episode.

State Normalization: To stabilize training, all observation variables, including UAV position, user locations, battery level, link-quality indicators, and task readiness, are normalized before being input to the Transformer-based actor-critic model. Normalization prevents large numerical variations from dominating the learning process and improves convergence by maintaining a consistent feature scale.

Reward Penalties and Dependency-Aware Factors: The reward function includes penalties for excessive delay, high energy consumption, and violations of task dependencies. Subtasks blocked by unfinished parents incur waiting penalties, while poor connectivity, excessive UAV movement, or low battery levels introduce additional costs. These mechanisms ensure that decisions respect communication feasibility and DAG execution order. By combining delay, energy, connectivity, and dependency effects into a unified reward signal, the agent learns policies that balance task completion efficiency with UAV energy conservation, consistent with the modelling principles of the base framework [6].

H. Problem Formulation for Energy-Delay Optimization

Unlike conventional UAV-assisted MEC models that assume independent tasks, the proposed formulation explicitly incorporates DAG-based task dependencies and integrates them with UAV mobility and communication constraints. Let $\xi_e(k, m) \in \{0, 1\}$ be a binary indicator denoting whether task $task_k^m$ is executed on computing device $e \in \mathcal{E} = \{0, 1, 2\}$, representing local IoT devices, UAV-mounted MEC, and Cloud server, respectively. The execution time and energy consumption are:

$$T(k, m) = \begin{cases} T_u(k, m), & \xi_0 = 1, \\ T_{u,a}(k, m), & \xi_1 = 1, \\ T_{u,s}(k, m), & \text{otherwise.} \end{cases} \quad (22)$$

$$E(k, m) = \begin{cases} E_u(k, m), & \xi_0 = 1, \\ E_{u,a}(k, m), & \xi_1 = 1, \\ E_{u,s}(k, m), & \text{otherwise.} \end{cases} \quad (23)$$

If $\xi_1 = 1$, the task is offloaded to the UAV-MEC, processed, and returned to the user. Task dependencies are modelled via the *Task Time Dependency* (TTD) constraint: a task can start only after all its predecessors finish, i.e.,

$$\tau_{k,l}^e \leq \tau_{k,m}^s, \quad \forall \text{con}_k(\text{task}_k^l, \text{task}_k^m). \quad (24)$$

The end time $\tau_{k,m}^e$ depends on the platform:

$$\tau_{k,m}^e = \begin{cases} \tau_{k,m}^s + T_u(k, m), & \xi_0 = 1, \\ \tau_{k,m}^s + T_{u,a}(k, m), & \xi_1 = 1, \\ \tau_{k,m}^s + T_{u,s}(k, m), & \xi_2 = 1. \end{cases} \quad (25)$$

with $\tau_{k,m}^s = \max\{\tau_{k,l}^e\}$ over all predecessors.

We aim to minimize total energy (including UAV flight E_f and hover E_h energy):

$$(\mathcal{P}_1) : \min_{\{\eta, \theta, \omega\}} \left(\sum_{\tau} [E_f(\tau) + E_h(\tau)] + \sum_{\tau, k, m, e} \xi_e(k, m) E_e(k, m) \right) \quad (26)$$

and total delay:

$$(\mathcal{P}_2) : \min_{\{\lambda, \theta, \omega\}} \sum_{\tau, k, m, e} \xi_e(k, m) T_e(k, m) \quad (27)$$

s.t. **C1** : $\xi_e(k, m) \in \{0, 1\}$,

C2 : $\sum_e \xi_e(k, m) = 1$,

C3 : $\sum_{\tau} [E_f(\tau) + E_h(\tau) + E_c(\tau)] \leq \bar{E}$, (28)

C4 : Mobility and communication constraints,

C5 : Task-dependency (TTD) constraint.

This forms a Mixed-Integer Nonlinear Programming (MINLP) problem, NP-hard due to binary assignments, nonlinear dynamics, and conflicting delay-energy trade-offs. We therefore reformulate $(\mathcal{P}_1)$ and $(\mathcal{P}_2)$ as a Markov Decision Process (MDP) to enable reinforcement learning (RL)-based optimization. The proposed algorithm has been designed, and results depicting the main objectives are depicted in the form of graphs and tables.

I. Transformer-Based Reinforcement Learning Design

We develop a Transformer-based RL framework for task offloading in UAV-assisted MEC. The system initializes the UAV count N_u and represents application workloads as a directed acyclic graph (DAG) $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where nodes correspond to computation tasks and edges represent dependency constraints. At time step t , the system state $s_t \in \mathbb{R}^d$ includes UAV-user information such as location, channel conditions, and queue states. The state is normalized to $\tilde{s}_t$ before being processed by the learning model to ensure stable training. The Transformer-based actor then maps $\tilde{s}_t$ to an action $a_t \in \mathcal{A}$, which jointly determines task offloading decisions and resource allocation. The choice of the Transformer architecture is motivated by the complex interactions among UAV mobility, wireless channel dynamics, and task dependencies in DAG-based workloads. Traditional neural architectures such as multilayer perceptrons (MLPs) or recurrent networks struggle to capture long-range dependencies across multiple system variables. Unlike MLP-based policies that process input features largely independently, the Transformer explicitly models cross-feature interactions through the self-attention mechanism, allowing the learning model to capture complex relationships among communication conditions, task dependencies, and resource availability. In contrast, the self-attention mechanism enables

the model to learn global relationships among users, communication conditions, and task states simultaneously. This ability to capture cross-feature dependencies makes the Transformer particularly suitable for dynamic UAV-assisted MEC environments where decisions depend on multiple interacting factors.

The Transformer encoder captures global contextual relationships among system features using the self-attention mechanism [13]. The scaled dot-product attention is defined as

$$\text{Attention}(M, R, C) = \text{softmax}\left(\frac{MR^\top}{\sqrt{d_r}}\right)C, \quad (29)$$

where $M = EW^M$, $R = EW^R$, and $C = EW^C$ denote the query, key, and value matrices obtained through linear projection of the input embedding E , and d_r is the key/query dimension. The encoder further employs multi-head attention to jointly model multiple representation subspaces:

$$\text{MHA}(M, R, C) = \text{Concat}(H_1, H_2, H_3, H_4)W_O, \quad (30)$$

where $H_i = \text{Attention}(MW_i^M, RW_i^R, CW_i^C)$ and W_O denotes the output projection matrix.

After executing action $a(\tau)$, the system transitions to state $s(\tau + 1)$ and produces reward $r(\tau)$ reflecting task delay and energy consumption. The policy π_θ is trained to maximize the discounted cumulative reward

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{\tau=0}^T \eta(\tau)r(\tau) \right], \quad (31)$$

where $\eta \in [0, 1)$ is the discount factor.

To further improve exploration and training stability, the framework adopts the maximum entropy reinforcement learning objective. The policy seeks to maximize

$$L(\pi) = \mathbb{E}_{(s(\tau), a(\tau)) \sim \rho^\pi} [R(\tau) + E(\tau)], \quad (32)$$

where $R(\tau)$ denotes the cumulative reward

$$R(\tau) = \sum_{t=0}^{t_{\text{sim}}} r(s(\tau), a(\tau)) \quad (33)$$

and $E(\tau)$ represents entropy regularization

$$E(\tau) = \epsilon H(\pi(\cdot|s(\tau))). \quad (34)$$

The policy entropy is defined as

$$H(\pi(\cdot|s(\tau))) = \mathbb{E}_{a \sim \pi(\cdot|s(\tau))} [-\log \pi(a|s(\tau))]. \quad (35)$$

The optimal policy is therefore

$$\pi^* = \arg \max_{\pi} L(\pi), \quad (36)$$

where the temperature coefficient ϵ controls the trade-off between reward maximization and policy randomness. Maximizing policy entropy improves exploration and prevents premature convergence to suboptimal solutions, enabling the agent to learn robust offloading strategies under dynamic UAV-IoT environments.

J. Problem Formulation for Quantization Optimization

For doing model quantization for lightweight Transformer-based framework, let us suppose there is a neural network with L layers. Each layer l in the neural network multiplies its weights w_l with activations A_{l-1} . To enable low-power, high-throughput inference on UAV-mounted Edge hardware, we quantize activations and weights, reducing w_l from `float32` to `int8`. Let A_{l-1}^β denote activations quantized to β bits. The low-bit operation $w_l \cdot A_{l-1}^\beta$ yields a performance gain $\delta\mathbb{P}_l^\beta$ and accuracy degradation $\delta\mathcal{L}_l^\beta$ relative to the full-precision case. Our objective is to maximize the total performance gain under a constraint on the degradation of total loss $\delta\mathcal{L}$. Introducing a binary variable χ_l^β indicating whether the layer l uses β -bit activations, the bit allocation problem is:

$$(\mathcal{P}_3): \max_{\{\chi_l^\beta\}} \sum_{l=1}^L \delta\mathbb{P}_l \quad (37)$$

$$\begin{aligned} \text{s.t. } \mathbf{C6}: & \sum_{l=1}^L \delta\mathcal{L}_l \leq \delta\mathcal{L}, \\ \mathbf{C7}: & \delta\mathbb{P}_l = \sum_{\beta} \chi_l^\beta \delta\mathbb{P}_l^\beta, \forall l, \\ & \delta\mathcal{L}_l = \sum_{\beta} \chi_l^\beta \delta\mathcal{L}_l^\beta, \forall l, \\ \mathbf{C8}: & \sum_{\beta} \chi_l^\beta = 1, \chi_l^\beta \in \{0, 1\}, \forall l. \end{aligned} \quad (38)$$

Here, (37) maximizes the cumulative performance gain achieved by applying low-bit quantization across all network layers. Constraint C6 limits the overall degradation in model accuracy by ensuring that the accumulated loss across all layers does not exceed the predefined threshold $\delta\mathcal{L}$. Constraint C7 defines the layer-wise performance gain and loss as weighted sums determined by the selected bit-width configuration for each layer. Constraint C8 enforces that exactly one bit-width configuration is selected for each layer through the binary decision variable χ_l^β .

1) *Para-Normalization Tuning*: Post-training quantization often fuses TL layers into preceding weights to reduce Multiply-Accumulate (MAC) operations. However, reduced bit-widths can shift internal statistics. We propose *Para-Normalization* (PN) to re-align statistics before re-fusion.

a) *Reconstructing TL Layers*: Given original TL parameters α_o , ψ_o , and κ , initialize reconstructed parameters $(\mu, \sigma^2, \alpha_r, \psi_r)$ as:

$$\mu = \psi_r = \psi_o, \quad \sigma^2 = \alpha_o^2, \quad \alpha_r = \sqrt{\alpha_o^2 + \kappa}, \quad (39)$$

yielding $TL_r(x) \approx x$:

$$TL_r(x) = \alpha_r \frac{x - \mu}{\sqrt{\sigma^2 + \kappa}} + \psi_r. \quad (40)$$

Then update μ and σ^2 using calibration data, without changing α_r or κ .

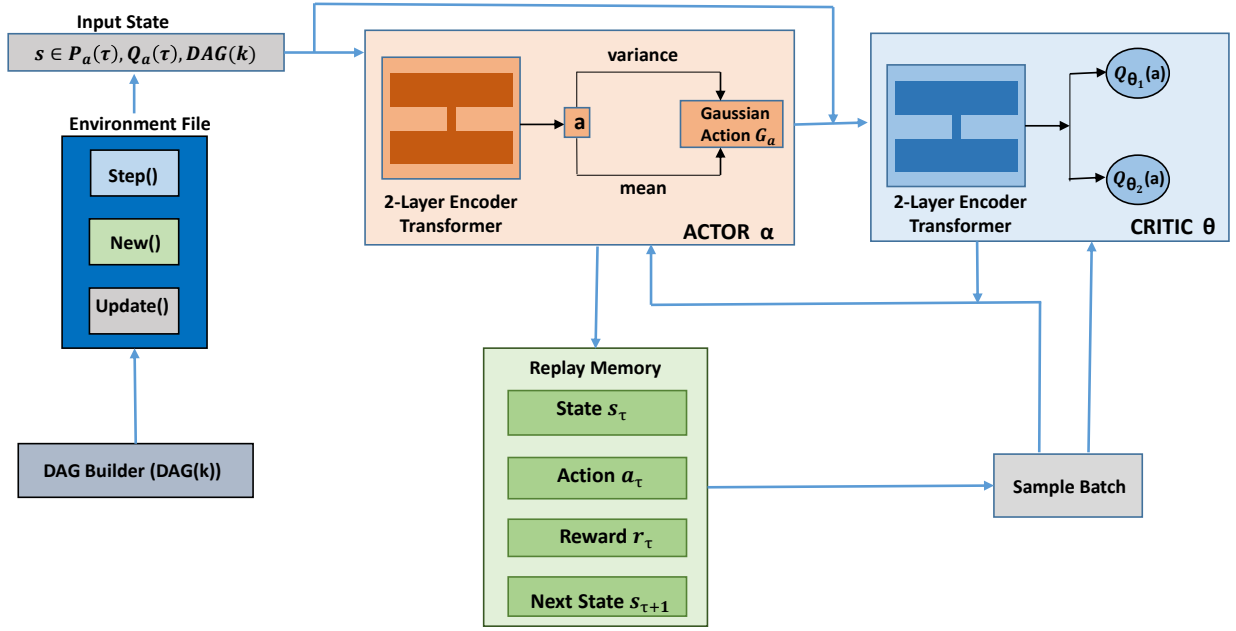


Fig. 3: Framework architecture for Quantized Transformer Based Energy Efficient Offloading.

b) *Re-fusing TL Layers*: In per-channel quantization, updated statistics are fused as:

$$w'_i = w_i \cdot \frac{\alpha_r}{\sigma}, \quad b'_i = \frac{\alpha_r}{\sigma}(b_i - \mu) + \psi_r, \quad \delta w'_i = \frac{\alpha_r}{\sigma} \delta w_i. \quad (41)$$

This ensures the quantization scale incorporates the PN adjustment. The problem of quantizing the 2-layer Transformer to make it deployable on a UAV-mounted Edge, enabling real-time inference within strict compute, memory, and energy budgets, is addressed in Section V.

IV. TRANSFORMER-BASED OFFLOADING AND QUANTIZED DEPLOYMENT FRAMEWORK

Fig. 3 shows the framework architecture that is built on the SAC-empowered UAV-assisted dependency-aware CO algorithm. The 2 major components, i.e. the actor and critic model that represent intelligence and UAV-assisted DAG-CO environment, are implemented using quantized transformers. The actor network receives the current observation state as input and generates the parameters (mean and variance) of a Gaussian-distributed policy to determine the optimal action. Meanwhile, the critic network takes an empirical transition tuple $(s_\tau, a_\tau, r_\tau, s_{\tau+1})$ as input and estimates the corresponding Q -value to evaluate the quality of the action. The subsequent procedural details are mentioned below.

A. Transformer Actor Module

a) *Policy Function*: The actor network represents the decision-making policy, mapping each observed system state to a corresponding action. In our framework, the Transformer encoder is used to capture dependencies among different elements in the state representation, which is essential for

effective offloading and resource allocation in a multi-user UAV-MEC setup.

b) *State Embedding*: The system state is first projected into a high-dimensional embedding space, where each dimension encodes a specific feature. This embedding is treated as a sequence of feature vectors, with the sequence length corresponding to the number of distinct state components. A learned positional encoding is added to these embeddings so that the Transformer can retain the relative ordering and positional relationships between state elements.

c) *Attention-Based Feature Extraction*: The embedded sequence passes through a two-layer Transformer encoder. Within the structure, each layer consists of two parts, multi-head self-attention and a feed-forward network. A two-layer architecture provides sufficient representational capacity to capture dependencies among system features while maintaining low computational complexity suitable for real-time inference on resource-constrained UAV platforms. This process enables the model to identify correlations across different features and to aggregate relevant information for action generation.

d) *Action Generation*: From the encoded feature representation, the network calculates Gaussian probability distribution from the encoded feature representation. It finds the mean and standard deviation for all possible actions.

e) *Differentiable Sampling*: Actions are sampled from this Gaussian distribution using a differentiable sampling technique (the reparameterization trick), which ensures that gradients can propagate through the sampling step during training. The actions are then passed through a hyperbolic tangent function to constrain them within the valid range for resource allocation decisions.

f) *Log-Probability Estimation*: The log-likelihood of the chosen action under the policy is computed, accounting for the transformation introduced by the squashing function. This term is later used in policy optimization to encourage actions that yield higher expected rewards.

B. Transformer Critic Module

a) *Value Estimation Role*: The critic module contains two separate Q-value estimators that evaluate the expected return for a given state-action pair. Using two critics helps reduce overestimation bias, which can otherwise destabilize training.

b) *Joint State-Action Embedding*: The state and the corresponding action are concatenated into a single vector and projected into the Transformer's embedding space. A positional encoding is then added to help the encoder process the combined input as an ordered sequence.

c) *Attention-Based Representation Learning*: The concatenated and encoded state-action sequence is processed by a two-layer Transformer encoder, enabling the critic to learn joint feature representations that capture interactions between state elements and the chosen action.

d) *Twin Q-Value Prediction*: The output of the Transformer encoder is fed into two independent prediction heads, each producing a separate estimate of the Q-value. In practice, the smaller of the two estimates is typically used during updates to improve stability.

C. Operational Workflow of the Transformer-Based Offloading Framework

The operational workflow of the proposed Transformer-based reinforcement learning framework for computation offloading in UAV-assisted MEC environments is illustrated in Fig. 4. The system begins by initializing the number of UAVs and modelling each application's tasks as a Directed Acyclic

Graph (DAG), where vertices represent computation tasks and edges denote task dependencies that must be respected during execution. At every time step, the system state includes UAV-user parameters such as physical locations, wireless channel quality, and queue status. This state vector is normalized before being processed to ensure stable and consistent learning. The Transformer-based actor module then maps the normalized state to an action, where each action specifies both the offloading decision (i.e., which computation node, i.e. UAV-MEC or Cloud, will handle the task) and the allocation of available computational and communication resources.

The overall decision pipeline operates as follows. IoT devices first generate application workloads represented as DAG structures. The system then constructs the environment state by combining task dependency information, communication conditions, and UAV resource availability. The normalized state representation is provided to the Transformer-based actor, which produces offloading and resource allocation decisions. The critic network evaluates the quality of the selected action, and the corresponding computation tasks are executed across local devices, UAV-mounted Edge servers, or the remote Cloud according to the chosen policy.

The Transformer encoder applies multi-head attention to model complex relationships within the input state features. Attention weights are computed by comparing query and key representations, scaled to avoid numerical instability, and used to weight the value representations. This enables the model to concentrate on the most relevant parts of the state when making decisions. The encoder also uses residual connections and feed-forward layers to refine contextual representations. Once an action is executed, the system transitions to a new state and computes a reward based on both the latency and the energy consumed in processing tasks. The objective is to maximize the expected cumulative reward over time, applying a discount factor between 0 and 1 to devalue future rewards. Policy optimization can be performed using either actor-critic or policy-gradient algorithms. By leveraging the Transformer's capability to capture both temporal and spatial correlations, the proposed design enables robust and adaptive decision-making in dynamic user scenarios.

V. ALGORITHM ANALYSIS

This section presents the algorithmic framework used to model DAG-aware task offloading in the UAV-assisted device-Edge-Cloud system. Algorithms 1, 2, and 3 respectively describe the environment dynamics, the reinforcement learning policy training, and the lightweight deployment of the quantized Transformer model.

A. Environment Dynamics: DAG-Aware UAV Task Offloading

Algorithm 1 describes the environment dynamics of the UAV-assisted dependency-aware task offloading system, which serves as the Markov Decision Process (MDP) for reinforcement learning. At each time frame τ , the system observes the global state s_τ , which includes the UAV position and remaining battery energy, IoT device locations, task queue states, and DAG dependency information. Based on this observation, the

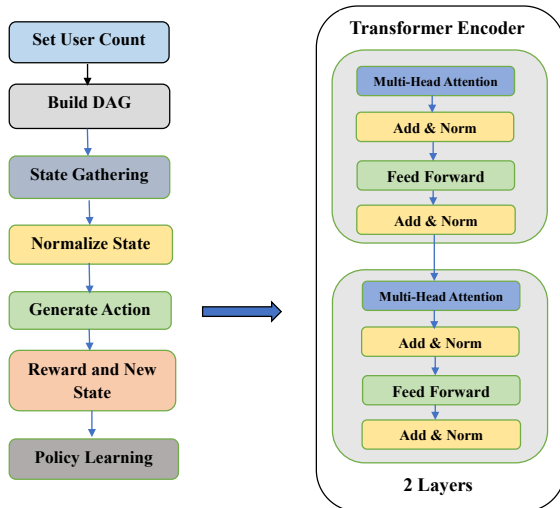


Fig. 4: Different States in Operational Workflow of Proposed Framework

Algorithm 1 Environment Dynamics: DAG-Aware UAV Task Offloading

```

1: Input: User set  $\mathcal{U}_\tau$ , DAG matrix task_depend,
   UAV/Cloud parameters
2: Initialize UAV battery  $E_a \leftarrow E_0$ , position  $\mathbf{P}_a$ , task list,
   user states
3: while tasks remain do
4:   Observe state  $s_t$ , agent selects  $a_t = [u, \theta, d, e \in \{0, 1, 2\}]$ 
5:   Update UAV position  $\mathbf{P}_a \leftarrow \mathbf{P}_a + (\theta, d)$ ; compute
   flight energy  $E_f$ 
6:   Check task parent dependencies from task_depend
7:   if parents incomplete then Penalize reward, update
   status, continue
8:   end if
9:   Compute  $(\tau_u, E_u), (\tau_a, E_a), (\tau_s, E_s)$  for
   local/UAV/Cloud
10:  Select mode  $\arg \min(0.5\tau + 0.5E)$ 
11:  if UAV out-of-bounds or low battery then Penalize,
   drop task, update state, continue
12:  end if
13:  Execute selected mode, update energy, latency, reward;
   mark task complete
14:  if all tasks done then set is_terminal  $\leftarrow$  True;
   break
15:  end if
16: end while

```

agent selects an action consisting of UAV movement parameters (heading and displacement) and an offloading decision indicating whether the task should be executed locally, at the UAV-mounted Edge server, or at the Cloud. After an action is selected, the UAV position is updated according to the movement command, and the corresponding flight energy consumption is calculated. The environment then checks the dependency constraints defined by the DAG structure to ensure that a task is executed only after all its parent subtasks are completed. If the dependency condition is violated, the task is blocked and the system assigns a penalty before proceeding to the next time step. For feasible tasks, the system evaluates execution latency and energy consumption under three execution modes: local device computation, UAV Edge execution, and Cloud processing. The mode that minimizes the weighted delay–energy cost is selected, and the system state is updated accordingly. Additional checks ensure that the UAV remains within the service region and maintains sufficient battery energy. If these conditions are violated, penalties are applied and the task may be dropped. The environment continues iteratively until all tasks are completed or the episode terminates. This dynamic interaction between system state, action selection, and environment response forms the basis for training the reinforcement learning policy.

B. Transformer-Based SAC Agent (2-Layer Encoder)

Algorithm 2 presents the training procedure of the Transformer-based reinforcement learning framework used for

Algorithm 2 Transformer-Based SAC Agent (2-Layer Encoder)

```

1: Initialize: Actor  $\pi_\phi$ , Critics  $Q_{\theta_1}, Q_{\theta_2}$ , Targets  $Q'_{\theta_1}, Q'_{\theta_2}$ ,
   Replay buffer  $\mathcal{B}$ , Temp.  $\alpha$ 
2: function ACTOR( $s$ )
3:    $x \leftarrow \text{Linear}(s) + \text{PosEmbed}$ 
4:    $z \leftarrow \text{TransformerEncoder}^{(2)}(x)$ 
5:    $\mu, \sigma \leftarrow \text{Linear}(z)$ ;  $a \sim \tanh(\mathcal{N}(\mu, \sigma^2))$ 
6:   return  $a, \log \pi(a|s)$ 
7: end function
8: function CRITIC( $s, a$ )
9:    $x \leftarrow \text{Linear}([s, a]) + \text{PosEmbed}$ 
10:   $z \leftarrow \text{TransformerEncoder}^{(2)}(x)$ 
11:  return  $Q_1(z), Q_2(z)$ 
12: end function
13: for each step do
14:  Observe  $s_\tau$ , select  $a_\tau \sim \pi_\phi(s_\tau)$ , store
    $(s_\tau, a_\tau, r_\tau, s_{\tau+1})$  in  $\mathcal{B}$ 
15:  if  $|\mathcal{B}| > \text{batch size}$  then
16:    Sample batch  $(s, a, r, s') \sim \mathcal{B}$ 
17:     $a', \log \pi' \leftarrow \pi_\phi(s')$ ,  $Q' \leftarrow \min Q'_{\theta_i}(s', a')$ 
18:     $y \leftarrow r + \gamma(Q' - \alpha \log \pi')$ 
19:     $\mathcal{L}_{\text{critic}} \leftarrow \sum_i \|Q_{\theta_i}(s, a) - y\|^2$ 
20:     $\mathcal{L}_{\text{actor}} \leftarrow \alpha \log \pi(a|s) - \min Q_{\theta_i}(s, a)$ 
21:     $\mathcal{L}_\alpha \leftarrow \alpha(-\log \pi(a|s) - \mathcal{H}_{\text{target}})$ 
22:    Update  $\theta_i, \phi$ , and  $\alpha$  using gradients
23:     $\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$ 
24:  end if
25: end for

```

optimizing dependency-aware computation offloading. At the beginning of each episode, the UAV position, user states, task queues, and DAG dependencies are initialized. During each time step, the agent observes the normalized system state containing UAV energy, user locations, link conditions, and task readiness information. The Transformer-based actor network processes this state representation through multi-head self-attention layers, allowing the model to capture relationships among task dependencies, wireless conditions, and resource availability. Based on the encoded features, the actor outputs the parameters of a Gaussian policy that generates the action, which determines both UAV movement and offloading decisions. After executing the selected action, the environment transitions to the next state and computes an immediate reward based on delay, energy consumption, and dependency violations. The transition tuple $(s_\tau, a_\tau, r_\tau, s_{\tau+1})$ is stored in the replay buffer. The critic network then evaluates the action value by estimating the Q-function, and the actor and critic parameters are updated using policy gradient optimization with entropy regularization. This process repeats across multiple episodes, enabling the policy to learn an energy-efficient offloading strategy under dynamic system conditions.

Algorithm 3 Dynamic Quantization of Transformer-Based Actor–Critic Networks

Require: Trained weights of actor and critic networks; state dimension d_s ; action dimension d_a

Ensure: Quantized actor and critic model weights

- 1: **Define:** File paths for original and quantized models
 - 2: Initialize actor model $\pi_\phi \leftarrow \text{TransformerActor}(d_s, d_a)$
 - 3: Initialize critic model $Q_\theta \leftarrow \text{TransformerCritic}(d_s, d_a)$
 - 4: Load trained parameters: $\phi \leftarrow \text{load}(\text{actor_path})$, $\theta \leftarrow \text{load}(\text{critic_path})$
 - 5: Set both models to evaluation mode
 - 6: Apply dynamic quantization to actor: $\pi_\phi^q \leftarrow \text{quantize_dynamic}(\pi_\phi, \{\text{Linear}\}, \text{dtype} = \text{qint8})$
 - 7: Apply dynamic quantization to critic: $Q_\theta^q \leftarrow \text{quantize_dynamic}(Q_\theta, \{\text{Linear}\}, \text{dtype} = \text{qint8})$
 - 8: Save quantized models: $\text{save}(\pi_\phi^q, \text{quantized_actor_path})$, $\text{save}(Q_\theta^q, \text{quantized_critic_path})$
 - 9: **Output:** Confirmation of successful quantization and deployment readiness
-

C. Dynamic Quantization of Transformer-Based Actor–Critic Networks

Algorithm 3 describes the post-training quantization process applied to the Transformer-based actor–critic networks to enable deployment on resource-constrained UAV hardware. The objective is to reduce computational complexity and memory requirements while maintaining decision accuracy. The algorithm begins with a trained full-precision neural network containing multiple Transformer layers. Each layer is analyzed to evaluate the trade-off between performance gain and accuracy degradation when activations and weights are quantized to lower bit-width representations. Binary decision variables are used to select the optimal quantization configuration for each layer while ensuring that the overall loss degradation remains within a predefined threshold. This optimization determines the bit-width assignment that maximizes inference efficiency under accuracy constraints. To improve numerical stability, Para-Normalization (PN) is applied before quantization to align the internal statistics of the Transformer layers. The normalization parameters are reconstructed and updated using calibration data, ensuring that the quantized model maintains similar activation distributions as the original network. Finally, the adjusted statistics are fused into the weight parameters, producing a compact INT8 model suitable for real-time inference on UAV-mounted Edge platforms.

VI. SIMULATION RESULTS

Extensive simulations are conducted to evaluate the performance of the proposed framework in a UAV-assisted device–Edge–Cloud collaborative computing environment. The proposed approach is compared with multiple baseline methods to assess its effectiveness in terms of system energy consumption, UAV energy usage, and task completion latency.

A. Simulation Settings

To evaluate the performance of the proposed Transformer-based UAV-assisted dependency-aware computation offloading

framework, we perform extensive simulations under controlled and realistic configurations. The entire environment is deployed in a $200\text{ m} \times 200\text{ m}$ square area, with a UAV flying at a constant altitude of 50 m and a maximum speed of 10 m/s. The UAV starts each episode with a finite energy budget and makes dynamic offloading and mobility decisions for each time frame. There are $M = 72$ tasks per instance, structured as a 4-layer DAG: (i) Start layer, (ii) Second and Third intermediate layers, and (iii) End layer. The task dependencies are deterministic and instance sizes vary among $\{6, 12, 18, 24\}$, creating different subtask counts per group. IoT users exhibit random-walk mobility within the deployment zone, and task generation follows frame-based arrivals with blocking propagation based on DAG status. Each episode spans multiple time frames, and models are trained over 1300 episodes using Soft Actor–Critic (SAC) with a Quantized Transformer backbone. Offline training is conducted with full-precision models, followed by post-training quantization for on-UAV deployment using 8-bit integer weights. The UAV performs inference onboard using quantized networks, while training is conducted offline.

B. Computational Complexity Analysis

This subsection summarizes the computational complexity of the three main components used in the proposed framework.

1) *Environment Dynamics (DAG-Aware UAV Task Offloading)*: In Algorithm 1, at each time step, the environment updates user states, UAV position, task dependencies, and energy–latency metrics. The per-step operations include constant-time state updates and DAG dependency checks of complexity $O(G)$, where G denotes the number of task edges in the DAG. For an episode containing E steps, the overall complexity is therefore $O(E)$.

2) *Transformer-Based SAC Agent*: In Algorithm 2, let d denote the model dimension, L the number of Transformer encoder layers, N the batch size, and T the total training iterations. With a small or constant sequence length, each forward pass of the actor or critic network requires $O(Ld^2)$

TABLE V: Simulation and Training Configuration

Parameter	Value
Simulation Area	200 m $\times$ 200 m
UAV Altitude	50 m
UAV Max Speed	10 m/s
Number of IoT Devices (M)	12 to 48
DAG Size per Instance	72 tasks, 4 layers
CPU Cycles per Bit (s)	500–1500 cycles/bit
Bandwidth Range	0.5–2 MHz
Replay Buffer Size	100,000
Batch Size	256
Actor/Critic Learning Rate	1e–4
Discount Factor (γ)	0.99
Soft Update Factor (τ)	0.005
Transformer Encoder Layers	2
Multi-head Attention Heads	4
Hidden Dimension	64
MLP Dimension	128
Quantization Type	Post-training INT8
Training Episodes	1300

operations. Consequently, the training complexity per step is $O(NLd^2)$ and the overall learning complexity across all iterations becomes $O(TNLd^2)$.

3) *Dynamic Quantization of Actor-Critic Networks*: In Algorithm 3, post-training dynamic quantisation is applied once after model training. If P represents the number of quantizable linear layers with weight size $d_{in} \times d_{out}$, the computational cost of quantization is $O(P d_{in} d_{out})$. Since this process occurs only once and does not affect inference-time execution, it introduces negligible runtime overhead during deployment.

C. Baseline Methods for Performance Comparison

To evaluate the effectiveness of the proposed framework, we compare its performance with commonly used reinforcement learning-based and heuristic baselines in UAV-assisted mobile Edge computing systems.

1) *UDCO-Based Model*: The UAV-assisted Dependency-Aware Computation Offloading (UDCO) model provides the primary baseline for comparison. This framework employs a device-Edge-Cloud collaborative computing architecture in which IoT tasks with DAG dependencies can be executed locally, at a UAV-mounted Edge server, or at the Cloud. The problem is formulated as a Markov Decision Process and solved using an improved actor-critic deep reinforcement learning algorithm to jointly optimize computation offloading decisions, UAV trajectory, and system cost considering delay and energy consumption [6].

2) *DDPG-Based Model*: The Deep Deterministic Policy Gradient (DDPG) algorithm is a model-free actor-critic reinforcement learning method designed for continuous control problems. In UAV-assisted MEC systems, it is commonly used to learn joint UAV trajectory and task offloading policies that minimize energy consumption and delay. In this work, the DDPG baseline follows the framework in [30], where the actor network generates continuous control actions and the critic network evaluates the corresponding Q-values.

3) *DRL-Based Model*: We also implement a generic Deep Reinforcement Learning (DRL) baseline that models the UAV-assisted task offloading problem as a Markov Decision Process (MDP). The DRL agent observes system states such as UAV position, user locations, and task conditions, and learns a policy that minimizes long-term delay and energy costs, following the approach in [31].

4) *Random Technique-Based Model*: As a non-learning benchmark, we include a random decision baseline in which task execution locations and offloading actions are selected randomly without considering system state or task dependencies. Similar random-assignment baselines have been used in UAV-assisted mobile computing studies such as [32].

D. Experimental DAG Task Instance Generation

To ensure fair and consistent evaluation across different algorithms, task instances used in the experiments are generated using a structured directed acyclic graph (DAG) workflow model. In all simulations, the total number of tasks is fixed as

$$M = 72. \quad (42)$$

Different task instances are created by varying the number of tasks within each group. The instance sizes used in the experiments are defined as

$$n \in \{6, 12, 18, 24\}.$$

For a given instance size n , the total number of task groups is calculated as

$$\text{group}_j = \frac{M}{n}. \quad (43)$$

The DAG is organized into four logical layers: a start layer, two intermediate layers, and an end layer. The start and end layers each contain group_j tasks, while the remaining tasks are evenly distributed across the two intermediate layers as

$$\text{inter_layer_nums} = \frac{M - 2 \times \text{group}_j}{2}. \quad (44)$$

Task dependencies are constructed to preserve sequential execution across the DAG layers. Each task in the second layer depends on one task from the start layer, tasks in the third layer depend on outputs from multiple second-layer tasks, and tasks in the final layer depend on the completion of third-layer tasks. Consequently, the resulting workflow follows the layered dependency structure.

$$\text{Start} \rightarrow \text{Second} \rightarrow \text{Third} \rightarrow \text{End}.$$

For example, when the instance size is $n = 6$, the number of task groups becomes $72/6 = 12$, and the remaining tasks are distributed between the two intermediate layers while maintaining the four-layer DAG structure used in the experiments.

E. Results Description

This subsection evaluates the performance of the proposed Transformer-based UAV-assisted Edge-Cloud computation offloading framework through extensive simulations. The obtained results are compared with baseline approaches to analyze improvements in energy consumption, task latency, training stability, and model efficiency.

Simulation Setup for Episode-based Evaluation: All energy-related results in Figures 5a through 5b are computed over 1300 episodes. The UAV starts each episode with a fixed energy budget, and trajectories evolve under episode-based reset conditions. Transformer-based models used in these simulations are quantized (INT8) for deployment feasibility and are pre-trained offline using full-precision SAC.

Fig. 5a presents the energy consumption per episode trajectories across the full episode horizon for five algorithms: DDPG-based ($\bar{E} = 33737.42$ J), DRL-based ($\bar{E} = 33167.10$ J), Random-based ($\bar{E} = 35262.14$ J), Transformer-based ($\bar{E} = 9888.25$ J), and UDCO-based ($\bar{E} = 12946.76$ J), where $\bar{E}$ = Average energy over 1300 episodes. While this global view captures overall trends, dense curve overlap, particularly in the 400–1200 episode window, reduces visual separability between methods, making it difficult to discern the Transformer's distinct low-energy profile despite its clear numerical advantage. The reduction in average energy consumption can be attributed to the Transformer's ability to

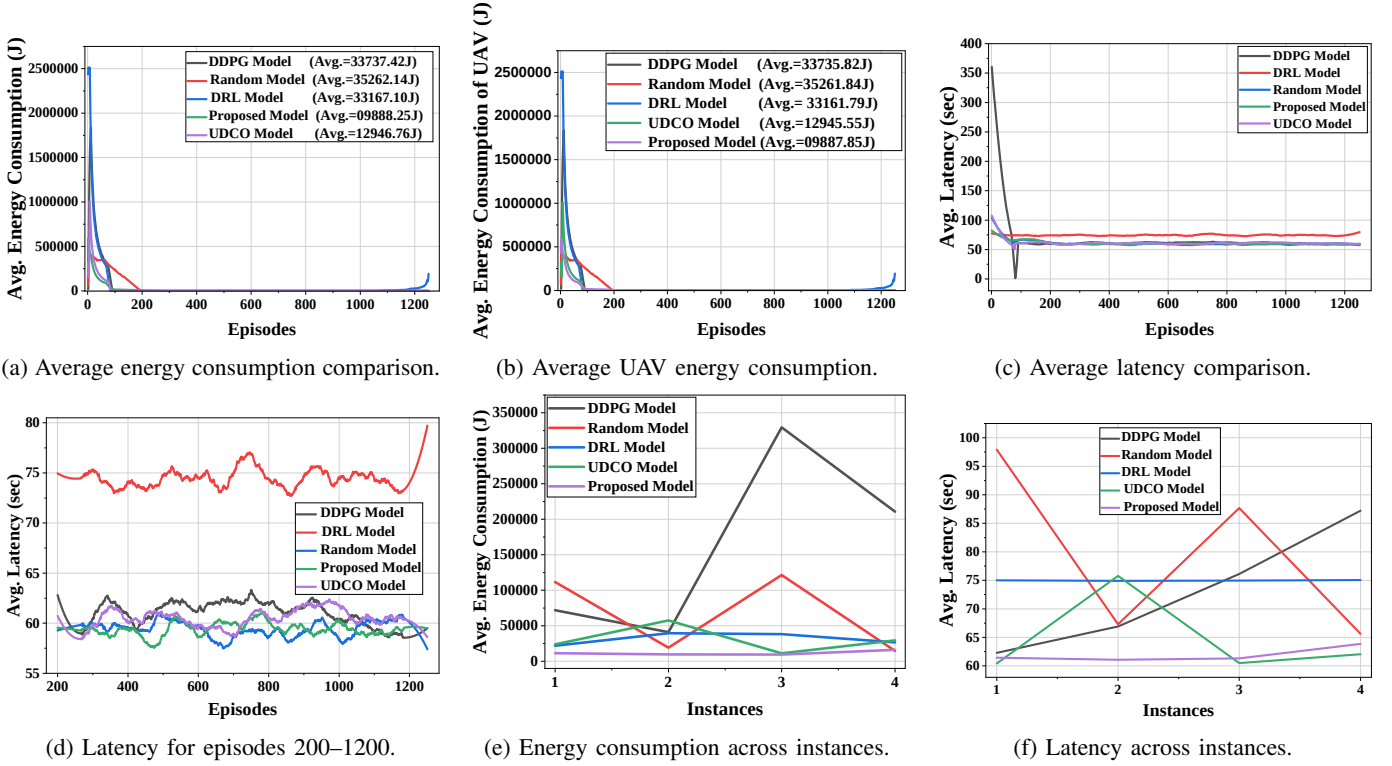


Fig. 5: Performance comparison of the proposed framework across different metrics and training stages.

effectively capture correlations and relative importance among a wide range of subtasks defined within the DAG, thereby enabling more efficient computation offloading.

Fig. 5b illustrates the UAV energy consumption per episode across for five algorithms: DDPG-based ($\bar{E}_{UAV} = 33735.82$ J), DRL-based ($\bar{E}_{UAV} = 33167.10$ J), Random-based ($\bar{E}_{UAV} = 35261.54$ J), UDCO-based ($\bar{E}_{UAV} = 12945.55$ J), and Transformer-based ($\bar{E}_{UAV} = 9887.848$ J), where $\bar{E}$ = Average energy over 1300 episodes. This global view highlights the Transformer’s substantial reduction in UAV energy demand compared to all baselines. However, dense curve overlap, especially in the convergence region, limits the ability to visually assess fine-grained performance differences. The similarity between the overall average energy and the average UAV energy highlights the optimal utilization of the UAV’s available computational resources, with only minimal computation performed on the IoT devices.

Latency Evaluation Conditions: System latency results are based on CPU cycle-per-bit values ranging from 500 to 1500 and a communication bandwidth fixed at 1 MHz unless stated otherwise. These variations model computational stress and link capacity fluctuation in UAV-assisted MEC, allowing assessment of real-time responsiveness under diverse load profiles.

Fig. 5c reports the system latency per episode, measured in seconds, for five algorithms: DDPG-based ($\bar{L} = 67.68995$ s), DRL-based ($\bar{L} = 74.48470$ s), Random-based ($\bar{L} = 61.11514$ s), UDCO-based ($\bar{L} = 61.39130$ s), and Transformer-based ($\bar{L} = 60.75266$ s), where $\bar{L}$ = Average Latency over 1300 episodes. Here, Latency encompasses

both computational and communication delays across the UAV and its supporting infrastructure for task execution. The Transformer-based approach achieves the lowest Latency among all methods, indicating superior end-to-end responsiveness, an essential factor for time-critical UAV-assisted MEC missions. The nearly identical average Latency across methods can be attributed to the fact that most of the delay arises from communication overhead rather than computation. The superior latency performance observed in the quantized Transformer case results from the use of vectorized INT8 operations, which execute significantly faster than their floating-point counterparts.

Fig. 5d narrows the analysis to episodes 200–1200, a range where convergence behaviour and stability in Latency become more distinguishable across methods. In this focused view, the Transformer-based model sustains its lead with $\bar{L} = 60.75266$ s, converging rapidly and exhibiting minimal variance throughout the steady-state phase. This low and stable latency profile confirms the Transformer’s capability to accelerate task completion and minimize communication delays, making it highly effective for UAV deployments requiring stringent real-time performance guarantees.

Instance Scaling Details: For Figs. 5e, 5f, 6a, 7a, and 7b, all simulations use a DAG of 72 tasks per instance. Instance sizes refer to how these tasks are grouped and layered (e.g., instance size 6 means 5 tasks per DAG layer). IoT user count is varied across 12, 30, and 48 to evaluate scalability and load tolerance.

Fig. 5e evaluates the scalability of each algorithm in terms of average energy consumption across varying instance

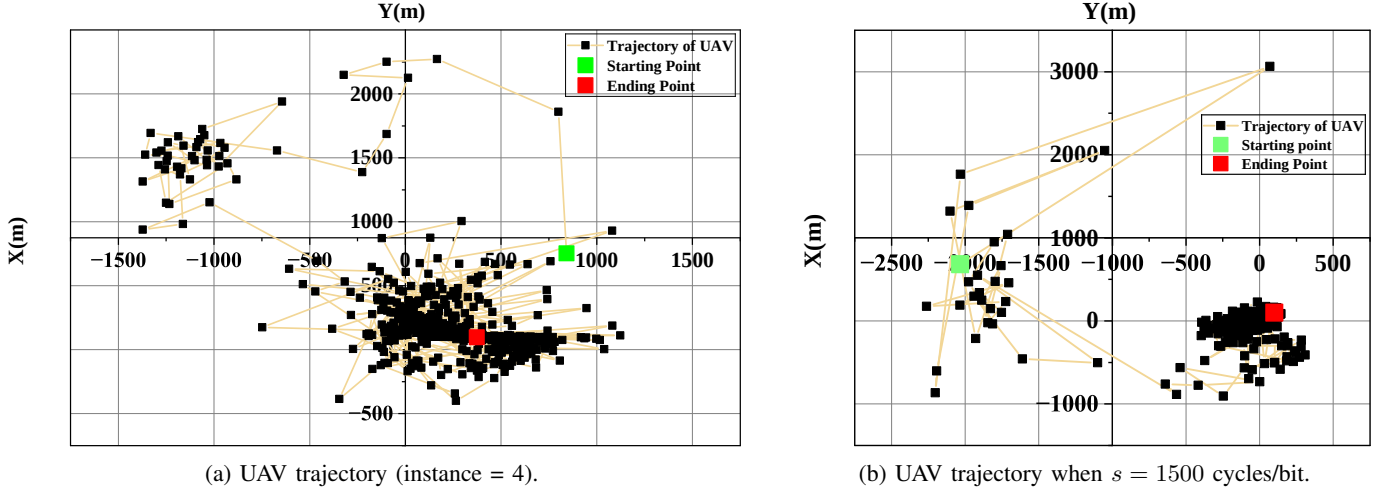


Fig. 6: Illustration of UAV trajectories under different system configurations.

sizes, where larger instances correspond to more complex DAG structures and increased inter-task dependencies. The Transformer-based approach consistently demonstrates strong energy efficiency and remains competitive with or superior to all baselines for most instance sizes. This is due to its ability to manage high-complexity workloads with minimal energy overhead, as it can see the inter-relations between the whole instances, i.e. the subtasks of a task taken. Notably, at instance sizes 3 and 4, the Transformer model exhibits a moderate increase in energy consumption relative to the UDCO algorithm. This deviation can be attributed to UAV trajectory adjustments required for task servicing under these configurations, resulting in longer travel paths and additional propulsion energy expenditure. Such behaviour highlights the interplay between scheduling decisions and UAV mobility patterns in determining system-level energy performance.

Fig. 5f compares the end-to-end latency performance of all algorithms as instance size varies. Here, Latency reflects the total time for computation and communication, encompassing UAV flight-induced delays and MEC processing times. The Transformer-based model consistently achieves the lowest Latency across all instance sizes, indicating rapid convergence and superior scheduling adaptability. Even as the DAG complexity increases and the UAV must dynamically adjust to changing user demands, the Transformer maintains responsiveness and minimizes task completion time, outperforming or matching the best baseline in every configuration. The combination of attention-driven task prioritization and robust decision-making under varying workloads enables the Transformer to sustain low-latency operation in highly dynamic UAV-assisted MEC environments. An interesting observation is that the baseline UDCO model slightly outperforms the proposed model for larger instance sizes. This marginal advantage can be attributed to the initialization randomness in data and the stochastic movement patterns of UAVs, which introduce minor fluctuations in communication latency that dominate the overall latency behaviour.

Fig. 6a shows the UAV movement trajectory for instance tag value as 4, which involves a more complex DAG (Di-

rected Acyclic Graph) task dependency structure. The location of the UAV changes frequently and stochastically, covering substantial distances between its starting and end points and traversing multiple regions of the deployment area with lateral and diagonal movements. In this case, the higher complexity of DAG handling requires more coordination in task scheduling, and when combined with the high mobility of the UAV, it results in longer average wireless transmission distances to certain users. This combination contributes to a noticeable increase in overall task offloading Latency, as the Transformer-based policy must adapt continuously to rapidly changing link conditions while managing intricate task dependencies. The UAV trajectory is visualized in a Cartesian coordinate system whose origin is located at the centre of the service region. Although the operational area is defined as a 200×200 m square, the coordinate system spans from -100 m to $+100$ m along both the x - and y -axes. This representation divides the service region into four quadrants and provides a clearer visualization of UAV movement relative to the centre of the coverage area. Therefore, the apparent four-quadrant layout in the trajectory figure does not indicate a larger simulation region but simply reflects the coordinate system used for plotting the UAV path. All UAV movements, therefore, remain within the 200×200 m service region defined in the system model.

Fig. 7a quantifies the variation in average energy consumption as the number of IoT devices scales from 12 to 30 and 48, with the instance size fixed at 1. Across all load points, the Transformer-based policy yields the minimum energy footprint relative to DDPG-, Random-, DRL-, and UDCO-based baselines. This consistent gap indicates more effective joint optimization of (i) UAV mobility (trajectory and hovering locations) and (ii) computation offloading decisions. In particular, the Transformer's attention mechanism improves temporal-state aggregation (task arrivals, queue backlogs, link qualities), enabling sharper predictions of near-future load and more energy-aware actuation; hence, it avoids myopic transmissions and unnecessary propulsion or compute spikes that typically inflate energy at higher densities. The pronounced

energy spike observed in the DDPG model can be attributed to its reliance on instantaneous state observations, without incorporating heuristic knowledge or inter-task dependencies. As a result, when dealing with correlated subtasks defined within the DAG, the agent frequently alternates between local computation and task offloading. These repeated switching behaviors lead to redundant communication and increased UAV propulsion activity, thereby elevating overall energy consumption.

Fig. 7b reports average Latency under the same scaling. While all methods incur higher delay as user density increases (due to rising compute and communication contention), the Transformer-based controller exhibits the smallest growth rate. By jointly regulating queue backlogs and flight path decisions, and by rapidly re-weighting state features via multi-head attention, it suppresses queueing and transmission delays even at 48 devices. This translates into lower task turnaround time without sacrificing energy efficiency.

Fig. 7c depicts the evolution of energy consumption, on average, as the communication bandwidth (unit is MHz as it is an analogue system) increases. All approaches, including DDPG, Random, DRL, UDCO, and the proposed Transformer technique, exhibit declining energy consumption due to communication time and overhead decreasing at increased available bandwidth. The proposed Transformer method consistently achieves lower energy consumption across the entire bandwidth. This is due to the ability to jointly optimize the allocation of tasks and the actual trajectory of the UAV while allowing the system to adopt a transmission strategy that exploits available channel capacities. The attention-based state representation the Transformer provides allows the system to predict where task execution constraints will occur and schedule task offloading in such a way as to limit time spent communicating and propulsion energy in completing the requested task, regardless of the surrounding network conditions. Again, high energy in DDPG accounts due to the primary reason of considering only current instances, not the relation between them.

Fig. 7d presents the relationship between CPU processing capability and system time delay (in seconds) for the same set of algorithms. All approaches exhibit decreasing Latency as CPU capability increases, reflecting faster execution of per-bit computational loads. The Transformer-based model achieves the most pronounced reduction, highlighting its ability to capitalize on enhanced processing power for improved responsiveness. However, at $s = 1500$ cycles/bit, the Transformer exhibits a noticeable latency increase relative to neighbouring configurations. This anomaly arises because the per-bit cycle count is substantially higher, meaning that even relatively small input data sizes incur significant total CPU cycles before completion. The resulting computation-heavy workload saturates onboard processing, delaying downstream scheduling and communication, and thus creating a localized performance bottleneck. This observation underscores the sensitivity of Latency to CPU-cycle-intensive configurations, even in otherwise high-efficiency Transformer-based MEC deployments.

Fig. 6b shows the UAV's movement trajectory for a computational density $s = 1500$ cycles/bit. Since the UAV is mobile,

it changes its position frequently, covering different quadrants of the $200 \text{ m} \times 200 \text{ m}$ operational area. While the operational area is set to $200 \text{ m} \times 200 \text{ m}$, the trajectory occasionally shows positions outside this range. This is due to the simulation allowing a temporary overshoot before boundary reset conditions are applied. As seen in the graph, the UAV travels considerable distances between steps. It not only travels from its starting point to the ending point but also through wide lateral and diagonal movements during the episode. This high mobility pattern, with rapid and large shifts in position, results in longer wireless transmission distances to certain users, which in turn slightly increases the overall task offloading Latency. The effect is more pronounced here because the computational density s is higher, causing the Transformer-based policy to process and adapt to rapidly changing link conditions more frequently.

Fig. 7e illustrates the variation in critic loss for the Transformer-based SAC model across the entire training horizon under different replay buffer capacities (5,000; 10,000; 20,000; 50,000; and 100,000). The results show a clear dependence of convergence behaviour on buffer size. Smaller buffers (e.g., 5,000) exhibit pronounced oscillations in the early training phase and slower convergence to a stable loss, likely due to reduced sample diversity and a higher correlation between stored transitions. Conversely, larger buffers ($\geq 50,000$) yield smoother and more monotonic loss decay, stabilizing at lower values—an indication of improved generalization in value estimation. This trend reflects the advantage of richer state-action coverage in large replay memories, which mitigates overfitting to recent experiences and enhances learning stability.

Fig. 7f provides a magnified analysis (episodes 200–1,200) of the critic loss under the same buffer configurations. In this range, the benefit of larger buffers becomes more evident. The loss curves for buffer sizes 50,000 and 100,000 rapidly settle into a low-volatility regime, whereas smaller buffers still show noticeable noise. The oscillation can also be attributed to the uneven error surface due to the large number of parameters present in transformers. This mid-training stabilization phase is critical for achieving high-quality value function estimates before fine-tuning policies in later stages. The reduced variance for larger buffers underscores their ability to maintain a balanced mix of short-term and long-term experiences during gradient updates.

Fig. 8a depicts the episode reward trajectories for different replay buffer sizes. Larger buffers (50,000 and 100,000) consistently outperform smaller ones, reaching higher peak rewards earlier and maintaining stability thereafter. In contrast, small buffers lead to more erratic reward progression, with frequent dips due to unstable critic estimates propagating to the actor updates. The superior performance of large buffers is attributed to their ability to store a broader distribution of state-action-reward tuples, ensuring that policy updates are less biased toward recent, potentially non-representative samples.

Fig. 8b examines the critic loss evolution under different discount factors ($\gamma \in 0.90, 0.95, 0.98, 0.99$). Lower values of γ (0.90, 0.95) produce more volatile loss curves and slower convergence, as the agent overemphasizes short-term rewards,

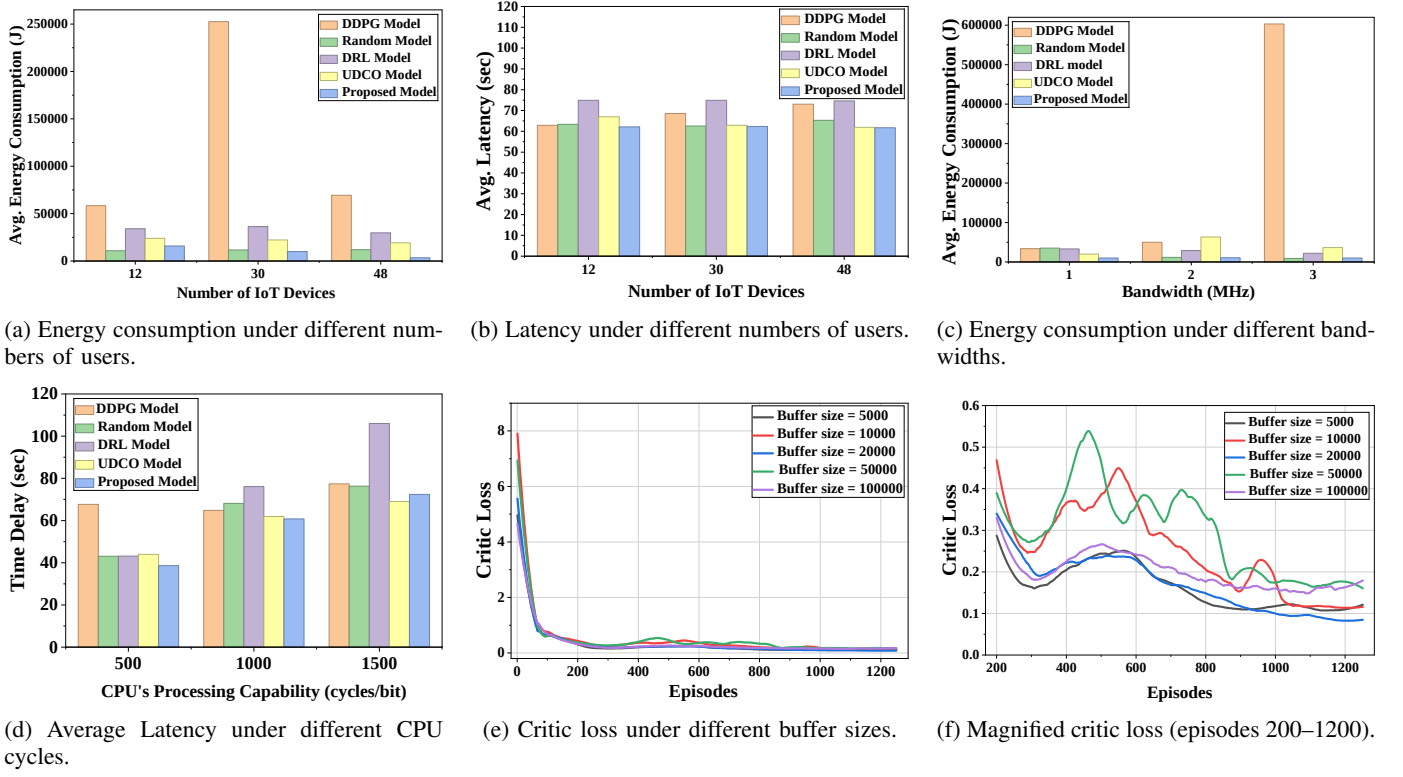


Fig. 7: Performance evaluation of the proposed framework under varying system parameters, including number of users, bandwidth availability, CPU computation capability, and training stability measured through critic loss.

causing higher variance in value targets. Higher values (0.98, 0.99) lead to markedly smoother loss trajectories, converging to lower loss plateaus—indicating that long-term reward consideration enhances the stability of value estimation and reduces bootstrapping error variance. The reason is clear, as the DAG-based task structure implies that the completion of the current subtask is highly dependent on the completion of its preceding subtasks. Consequently, greater emphasis must be placed on past rewards, which justifies the use of a higher discount factor (γ), giving better results.

Fig. 8c presents the corresponding episode reward trends for varying γ . The patterns confirm that high discount factors accelerate reward growth and sustain high performance over time, with $\gamma = 0.99$ yielding the most consistent and highest rewards. Lower γ values underperform, with slower ascent and larger oscillations. This reinforces the role of long-term foresight in policy optimization, encouraging strategies that may incur short-term sacrifices for higher cumulative returns. Due to the lower critic loss associated with higher γ as we discussed in previous graph, the resulting reward becomes smoother and higher.

F. Justification for Quantized Transformer Models on UAV Systems

UAV platforms have stringent limits on storage, memory, and processing power, making full-precision Transformers impractical. Quantization reduces model size and memory usage without altering architecture or decision fidelity, enabling efficient deployment on embedded processors.

Table VI summarizes the hardware requirements of the proposed Quantized Transformer models compared to their non-quantized counterparts. The quantized versions retain identical parameter counts and architecture while significantly reducing model size (from 10.18 MB to 4.09 MB) and RAM usage (from 0.22 MB to 0.18 MB), demonstrating their suitability for UAVs with constrained storage and memory resources.

Storage and Memory Gains: The compression ratio is:

$$\frac{S_{\text{float}} - S_{\text{int8}}}{S_{\text{float}}} \approx 59.8\% \quad (45)$$

reducing model size from 10.18 MB to 4.09 MB, enabling storage on UAV hardware with <32 MB flash and faster over-the-air updates. RAM usage drops from 0.22 MB to 0.18 MB:

$$\text{Reduction} \approx 18.18\% \quad (46)$$

supporting real-time inference within ≤ 512 MB RAM constraints.

Fig. 8d compares the total storage footprint of the Actor and Critic networks for both baseline full-precision and post-training quantized Transformer models. The baseline models occupy approximately 10 MB, whereas their quantized counterparts require only 4 MB, representing an $\approx 60\%$ reduction in storage. This substantial compression stems directly from reduced-precision weight representation (e.g., FP32 $\rightarrow$ INT8), which minimizes non-volatile memory requirements and accelerates model loading from storage. Such savings are critical for UAV-assisted MEC platforms, where persistent storage is severely constrained, and multiple models or auxiliary mission software must coexist onboard.

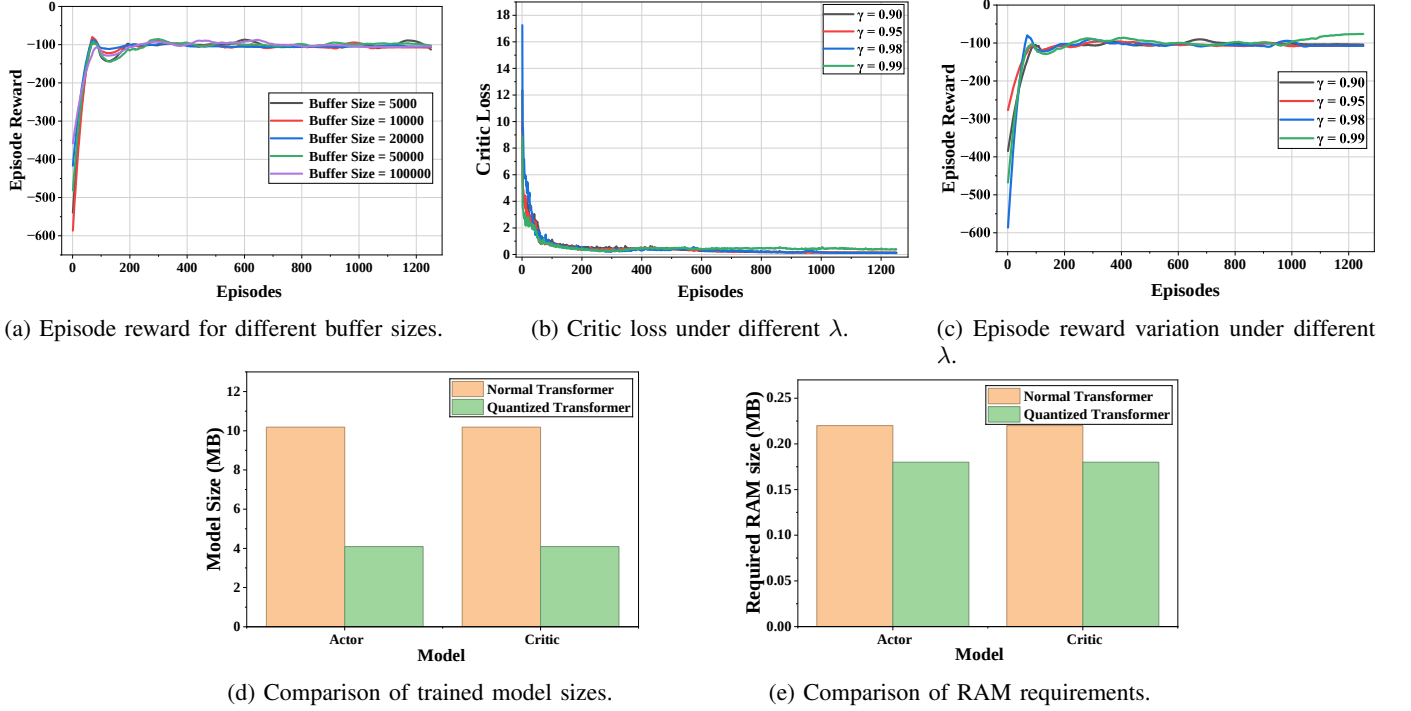


Fig. 8: Impact of buffer size and discount factor on training performance, along with model compression results in terms of model size and RAM requirements.

TABLE VI: Quantized vs. Non-Quantized Transformer Hardware Requirements

Model	Size (MB)	#Params	Enc. Layers	Linear	Scales	ZPs	RAM (MB)
TransformerActor	10.18	271,828	2	6	0	0	0.22
TransformerCritic	10.18	310,212	2	8	0	0	0.22
QuantizedActor	4.09	271,828	2	6	16	16	0.18
QuantizedCritic	4.09	310,212	2	8	18	18	0.18

TABLE VII: Hardware footprint comparison for Transformer models in UAV deployment

Metric	Full-Precision	Quantized	UAV Limit
Model Size	> 10 MB	4.09 MB	4–8 MB flash
RAM Usage (per module)	220 KB	180 KB	< 1 MB MCU / 2–4 GB SoC
Compute Efficiency	Floating-point operations	$\times 4$ speedup (INT operations)	Real-time capable

Architectural Fidelity and Speedup: Parameter counts and encoder layers remain unchanged, ensuring $\mathcal{F}_{\text{int8}}(x) \approx \mathcal{F}_{\text{float}}(x)$ with negligible accuracy loss. Quantization applies:

$$x_q = \text{round}\left(\frac{x}{s}\right) + z$$

enabling integer inference with a theoretical speedup:

$$\frac{C_{\text{float32}}}{C_{\text{int8}}} = 4 \times \quad (47)$$

Per-layer scales and zero-points (Actor: 16, Critic: 18) maintain functional precision across layers.

Fig. 8e presents the peak RAM utilization during inference for both the Actor and Critic modules. The baseline full-precision Transformer models require approximately 0.22 MB (220 KB), whereas their post-training quantized counterparts reduce this footprint to 0.18 MB (180 KB), achieving an $\approx 18\%$ reduction in memory consumption. This decrease is non-trivial in UAV-assisted MEC deployments, where onboard

DRAM is highly constrained and shared among multiple computational threads. By lowering parameter precision (e.g., FP32 $\rightarrow$ INT8), quantization compresses weight tensors and intermediate activations, thereby improving cache locality, reducing memory bandwidth requirements, and sustaining deterministic real-time inference within the strict memory and power budgets of embedded aerial platforms.

Quantized Transformer models align well with the hardware constraints of embedded UAV platforms, where MCUs often provide < 4 MB flash and < 1 MB RAM, and edge-grade SoCs up to 2–4 GB RAM. Full-precision Transformers, requiring > 10 MB storage and ~ 220 KB RAM per module, exceed these limits and often need external memory or GPU acceleration, which is impractical for low-power UAVs. In contrast, our quantized models fit seamlessly within onboard resources, as shown in Table VII.

Table VII shows that the quantized models not only meet but also exceed real-world UAV AI hardware constraints. They are

fully compatible with embedded inference frameworks such as TensorFlow Lite Micro, CMSIS-NN, and TVM, eliminating the need for expensive high-end CPUs or accelerators. The quantized Transformer achieves a 59.8% reduction in storage size and 18% lower RAM usage while preserving the same parameter count and architecture, making it nearly twice as efficient with no loss in functional capability, ideal for UAV platforms where energy and space are critical.

VII. CONCLUSION

This paper proposed a quantized Transformer-based reinforcement learning framework for dependency-aware computation offloading in UAV-assisted edge-cloud computing systems. The framework models DAG-based IoT workloads and uses a Transformer-enhanced actor-critic architecture to capture task dependencies and system dynamics. To enable deployment on resource-constrained UAV platforms, the trained policy is compressed using INT8 post-training quantization. By integrating dependency-aware scheduling with efficient Transformer-based decision making, the proposed framework provides a practical pathway toward intelligent task orchestration in next-generation UAV-assisted IoT edge computing systems. Simulation results show that the proposed method reduces total system energy consumption by 23.63%, UAV energy consumption by 23.62%, and average task latency by 1.04% compared with the baseline framework. The quantized model further reduces storage size by 59.8% and RAM usage by 18% while preserving decision performance. Although the current study relies on simulation-based evaluation, future work may explore real UAV testbed implementation, cooperative multi-UAV systems, and advanced model compression techniques for large-scale UAV-assisted MEC deployments. These results highlight the potential of combining dependency-aware learning with lightweight model quantization to enable practical intelligent computation offloading in UAV-assisted edge-cloud computing systems.

REFERENCES

- [1] L. Atzori, A. Iera, and G. Morabito, "The internet of things: A survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.
- [3] X. Xia, S. M. M. Fattah, and M. A. Babar, "A survey on uav-enabled edge computing: Resource management perspective," *ACM Computing Surveys*, vol. 56, no. 3, pp. 1–36, 2023.
- [4] B. Wang, H. Kang, J. Li, G. Sun, Z. Sun, J. Wang, and D. Niyato, "Uav-assisted joint mobile edge computing and data collection via matching-enabled deep reinforcement learning," *IEEE Internet of Things Journal*, 2025.
- [5] X. Liu, Z.-Y. Chai, Y.-L. Li, Y.-Y. Cheng, and Y. Zeng, "Multi-objective deep reinforcement learning for computation offloading in uav-assisted multi-access edge computing," *Information Sciences*, vol. 642, p. 119154, 2023.
- [6] L. Zhang, R. Tan, Y. Zhang, J. Peng, J. Liu, and K. Li, "Uav-assisted dependency-aware computation offloading in device-edge-cloud collaborative computing based on improved actor-critic drl," *Journal of Systems Architecture*, vol. 154, p. 103215, 2024.
- [7] Y. Yang, Y. Shi, X. Cui, J. Li, and X. Zhao, "A hybrid decision-making framework for uav-assisted mec systems: Integrating a dynamic adaptive genetic optimization algorithm and soft actor-critic algorithm with hierarchical action decomposition and uncertainty-quantified critic ensemble," *Drones*, vol. 9, no. 3, p. 206, 2025.
- [8] Y. Wang, W. Fang, Y. Ding, and N. Xiong, "Computation offloading optimization for uav-assisted mobile edge computing: a deep deterministic policy gradient approach," *Wireless Networks*, vol. 27, no. 4, pp. 2991–3006, 2021.
- [9] S. A. Zakarya, M. Meaad, T. Nabil, and M. K. Hussein, "Task offloading and resource allocation for multi-uav asset edge computing with multi-agent deep reinforcement learning," *Computing*, vol. 107, no. 5, p. 126, 2025.
- [10] W. Li, S. Li, H. Shi, W. Yan, and Y. Zhou, "Uav-enabled fair offloading for mec networks: a drl approach based on actor-critic parallel architecture: W. li et al." *Applied Intelligence*, vol. 54, no. 4, pp. 3529–3546, 2024.
- [11] Z. Wang, M. Goudarzi, and R. Buyya, "Tf-ddrl: A transformer-enhanced distributed drl technique for scheduling iot applications in edge and cloud computing environments," *IEEE Transactions on Services Computing*, vol. 18, no. 2, pp. 1039–1053, 2025.
- [12] Y. Li, C. Tang, B. Hou, F. Jiang, and Y. Fu, "Transformer and meta-reinforcement learning-based task offloading for mec systems in dynamic environments," *IEEE Access*, 2025.
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [14] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale," *Advances in neural information processing systems*, vol. 35, pp. 30 318–30 332, 2022.
- [15] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International conference on machine learning*. PMLR, 2023, pp. 38 087–38 099.
- [16] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, "I-bert: Integer-only bert quantization," in *International conference on machine learning*. PMLR, 2021, pp. 5506–5518.
- [17] C. Peng, Q. Wang, and D. Zhang, "Efficient dynamic task offloading and resource allocation in uav-assisted mec for large sport event," *Scientific Reports*, vol. 15, no. 1, p. 11828, 2025.
- [18] S. Xu, Q. Liu, C. Gong, and X. Wen, "Energy-efficient multi-agent deep reinforcement learning task offloading and resource allocation for uav edge computing," *Sensors*, vol. 25, no. 11, p. 3403, 2025.
- [19] V. Pourakbar and H. Shah-Mansouri, "Enhanced ai as a service at the edge via transformer network," in *2025 IEEE Wireless Communications and Networking Conference (WCNC)*. IEEE, 2025, pp. 1–6.
- [20] A. Ndiaye, M. Dieye, W. Jaafar, F. Balde, and R. Glitho, "A novel transformer reinforcement learning-based nfv service placement in mec networks," *Computing&AI Connect*, vol. 2, no. 1, pp. 1–10, 2025.
- [21] M. Nandagopal, T. Manavalan, K. P. Kumar, N. Manogaran, D. Kesavan, G. Kumar, and M. A. Al-Khasawneh, "Enhancing energy efficiency in cloud computing through task scheduling with hybrid cuckoo search and transformer models," *Discover Computing*, vol. 28, no. 1, p. 199, 2025.
- [22] Y. Lin, T. Zhang, P. Sun, Z. Li, and S. Zhou, "Fq-vit: Post-training quantization for fully quantized vision transformer," *arXiv preprint arXiv:2111.13824*, 2021.
- [23] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of machine learning and systems*, vol. 6, pp. 87–100, 2024.
- [24] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "Zeroquant: Efficient and affordable post-training quantization for large-scale transformers," *Advances in neural information processing systems*, vol. 35, pp. 27 168–27 183, 2022.
- [25] K. Dorling, J. Heinrichs, G. G. Messier, and S. Magierowski, "Vehicle routing problems for drone delivery," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 47, no. 1, pp. 70–85, 2016.
- [26] X. Xu, K. Liu, P. Dai, F. Jin, H. Ren, C. Zhan, and S. Guo, "Joint task offloading and resource optimization in noma-based vehicular edge computing: A game-theoretic drl approach," *Journal of Systems Architecture*, vol. 134, p. 102780, 2023.
- [27] I. Budhiraja, N. Kumar, H. Sharma, M. Elhoseny, Y. Lakys, and J. J. Rodrigues, "Latency-energy tradeoff in connected autonomous vehicles: A deep reinforcement learning scheme," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 11, pp. 13 296–13 308, 2022.
- [28] G. Wu, Z. Xu, H. Zhang, S. Shen, and S. Yu, "Multi-agent drl for joint completion delay and energy consumption with queuing theory in mec-based iiot," *Journal of Parallel and Distributed Computing*, vol. 176, pp. 80–94, 2023.

- [29] L. Zhang, D. Liu, M. Ai, R. Tan, and Z. Zeng, "Reliability enhancement algorithm based on budget level in cloud-edge environments," *International Journal of Embedded Systems*, vol. 16, no. 1, pp. 9–22, 2023.
- [30] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and A. Nallanathan, "Deep reinforcement learning based dynamic trajectory control for uav-assisted mobile edge computing," *IEEE Transactions on Mobile Computing*, vol. 21, no. 10, pp. 3536–3550, 2021.
- [31] L. Zhang, Z.-Y. Zhang, L. Min, C. Tang, H.-Y. Zhang, Y.-H. Wang, and P. Cai, "Task offloading and trajectory control for uav-assisted mobile edge computing using deep reinforcement learning," *IEEE Access*, vol. 9, pp. 53 708–53 719, 2021.
- [32] Q. Wang, A. Gao, and Y. Hu, "Joint power and qoe optimization scheme for multi-uav assisted offloading in mobile computing," *IEEE Access*, vol. 9, pp. 21 206–21 217, 2021.